LECTURE 04

版本、协作与需求演化

04

保存改变的来路，协调并行的工作，用真实需求检验架构。

19 节 · 35 幅配图 · 7 道自测　回看课堂 ↗

本课98分29秒；完整阅读20个最终转写块、463个合并句，保留原始导出与全部时间索引。原始转写含错词，正文据上下文、讲义和画面校正，未逐秒人工回听。

完整阅读官方课程正文与25页讲义文字；全部25页图已逐图目视。讲义以当前官网截图汇编PDF冻结，非官方原生PDF，也不标成同秒视频帧。

视频按5分钟网格与演示定点抽帧核对；未逐秒观看。屏幕中的Agent设计和自报结果，不等于本次运行或审计了其项目。

技术校注另核对jj与Git官方文档相关段落；其他课堂职业趋势、商业化和未来工具设想保留为讲师观点。

窄屏浏览器验收记录见 review.json；未做真实手机设备测试。

阅读导引

先把握本课主线

01 版本管理保存的不只是代码快照，还包括修改身份、**演化路径**与决策教训；文件齐全并不保证需求、文档、代码和测试之间的关联齐全。

02 Agent 加速生成以后，工作隔离、任务分解、**协调时机**、验证和整合必须一起改进。Worktree隔离工作目录，锁协调共享状态，两者都不自动保证正确。

03 从零开始时，早期提示会把含混意图迅速固定成架构。先用MVP验证真正需要什么，再用真实需求变化检验设计，比一次生成完整信息系统更容易保留选择空间。

第 01 节 00:00:00—00:05:21 · 课堂时间

Lab 1：有用的个人助手，也是一场维护实验

> **关键判断 · 整理者提炼**
> 持续工作的助手既要满足当下需要，也要经受后续需求变化，保留必要的人类确认。

本课继续版本管理，先发布选做实验：**会成长的个人助手**。实现方式不限，讲师鼓励直接动手，并计划当堂从实验要求开始生成一个版本。课程所说的个人数据库不限于关系数据库，可以是课程资料、论文和教科书，也可以是自己的履历、姓名学号等反复填表所需的信息。



资料图 · 第 1 页 复习与 Lab 1：会成长的个人助手。

能力从资料库向外延伸：关联 smail 邮箱、持续接收邮件并利用上下文处理、办理 eHall 事务、通过手机发起任务与查看进度。讲师设想把 Agent Loop 放在一台持续运行的小设备上；设备价格和自动化能力是课堂举例，笔记没有核实采购成本或可用性。

> **整理者说明**
> 官方作业画面明确：草稿可编辑后发送，提交 eHall 前应**确认关键字段**，缴费、撤销申请等重大操作不能由 Agent 自行决定。持续运行不等于取消人的确认权。

这个实验的意义并不只是“AI 能不能写出来”。直接把全部要求交给模型，它会立即作出一连串看似合理的决定。这些决定日后可能使维护越来越困难，修改消耗大量 token，最后迫使人回退、重构或重做。讲师预告后续 Lab 会追加与已有功能广泛关联且有冲突的需求，让大家亲身体验**需求变化**和**焦油坑**。

> **整理者说明**
> 转写中的 webcode 在上下文中指 Vibe Coding；一号事务按画面校正为 eHall 事务，AISLAP 按语境校正为 AI slop。没有回听确认的词不作为逐字引语。

第 02 节　　00:05:21—00:10:35 · 课堂时间

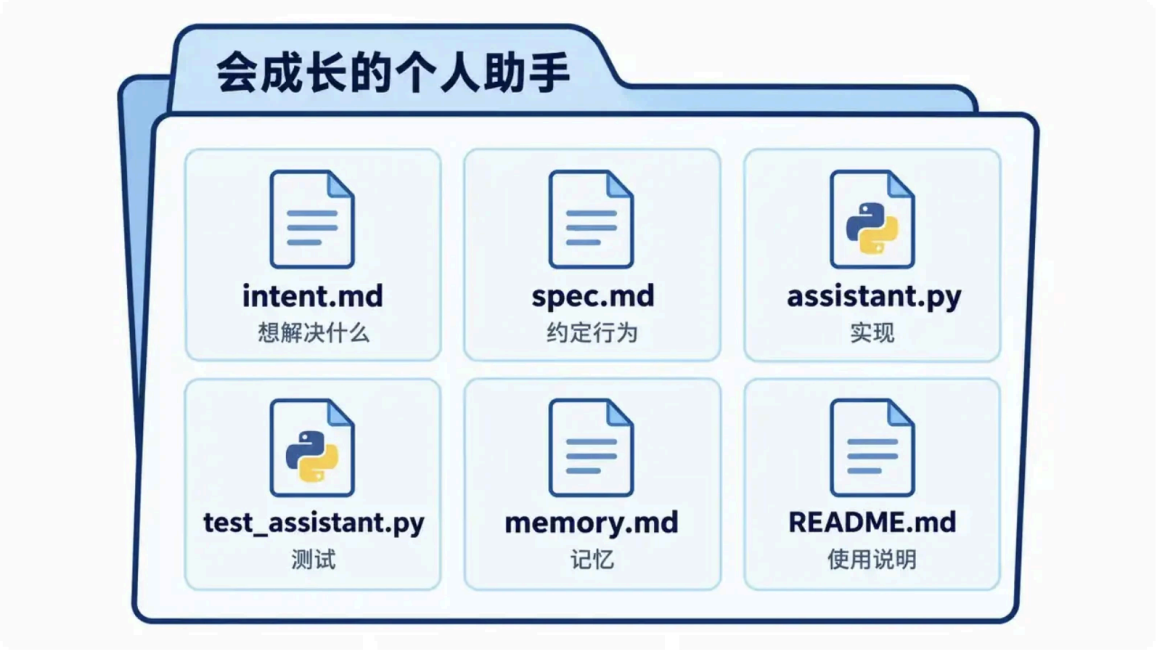
项目的文件齐全，不等于关系齐全

> **关键判断 · 整理者提炼**
> 可追溯性需要留下需求、实现与测试的语义关联，文件齐全本身不保证一致。

项目包含意图、设计规约、代码、测试与说明。讲师偏好的 README 很短：说清最初想做什么产品、想达到什么效果、最重要的设计，必要时配截图。它像向合作者说明一个值得做的点子，剩余的架构和实现可以由团队补齐；这不是要求把所有接口规约都塞进 README。

项目是各种文件的集合

Intent、Spec、Code、…



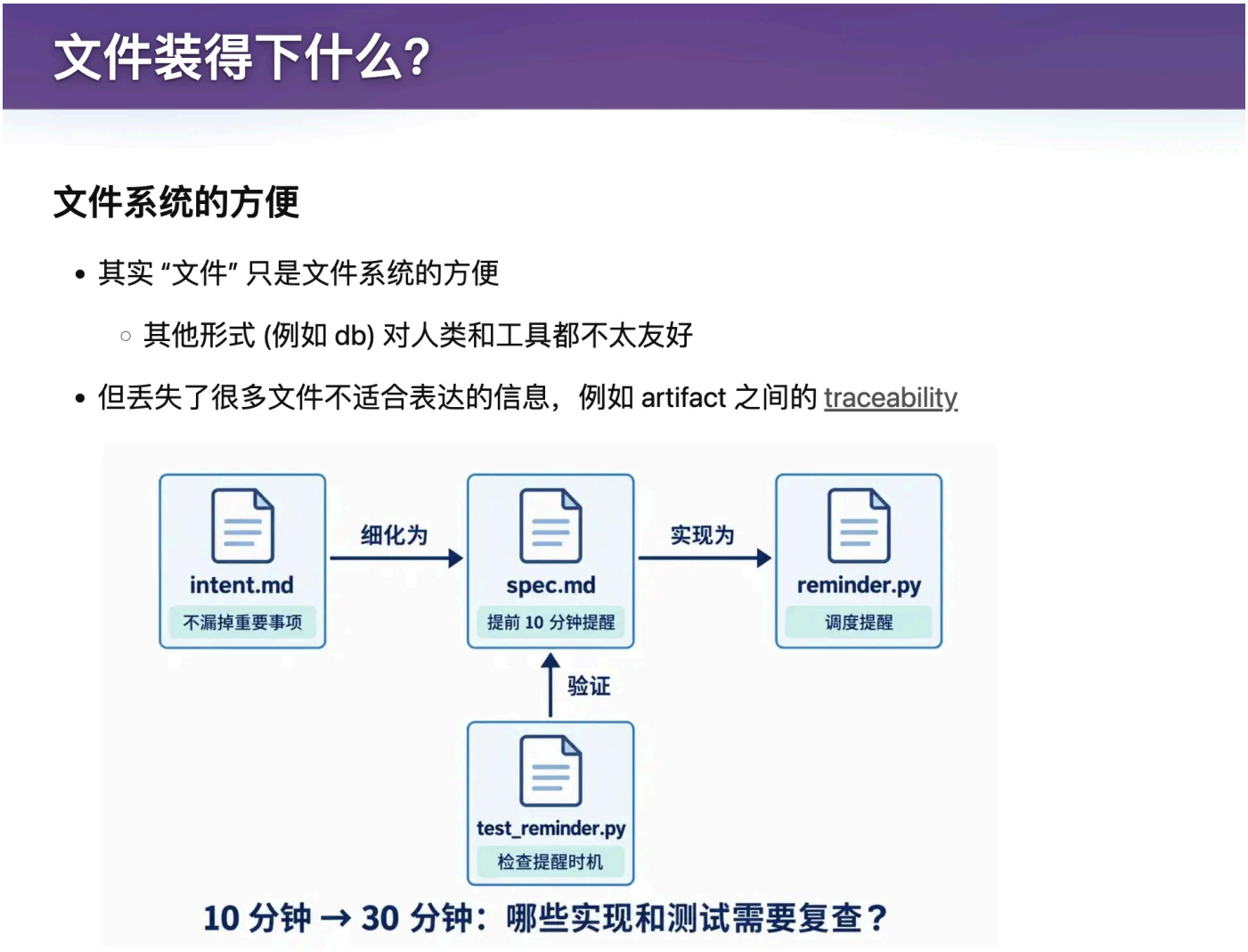
资料图 · 第 2 页 项目文件包含意图、规约、实现、测试、记忆与说明。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 2 · 原始讲义 · 项目文件包含意图、规约、实现、测试、记忆与说明。

文件系统提供现成容器，工具也围绕文件发展。但目录能保存一段需求、一份实现和一个测试，却不会自动保存三者的**依赖关系**。课堂例子是把链表换成性能更好的树：代码改完，文档还写“用链表管理”，测试也可能依赖旧假设。最初随手写的描述逐渐变成**技术债**。

当聊天记录消失、文档和实现各自演变，后来的开发者或 Agent 很难判断“文档错还是代码错”。**可追溯性**要求把设计、实现、测试之间的关联明确留下，使某项决定发生变化时，能找出需要一起检查的产物。文件并非不能记录这些信息，而是文件系统自身不会替项目建立这种**语义联系**。

> **整理者说明**
>
> 讲义补充的提醒器示例：意图“不漏掉重要事项”→规约“提前10分钟提醒”→实现 reminder.py，测试验证提醒时机。10改30时，需沿依赖找到实现和测试，而不是只改一句文字。



资料图 · 第 3 页 文件之外的关联：修改提醒时间需要复查哪些实现和测试?

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 3 · 原始讲义 · 文件之外的关联：修改提醒时间需要复查哪些实现和测试?

> **整理者说明**
> 讲师把文件组织方式解释为历史路径依赖；关于早期文件系统与数据库的具体年代，本笔记未另做计算机史核查。

第 03 节 00:10:35—00:19:15 · 课堂时间

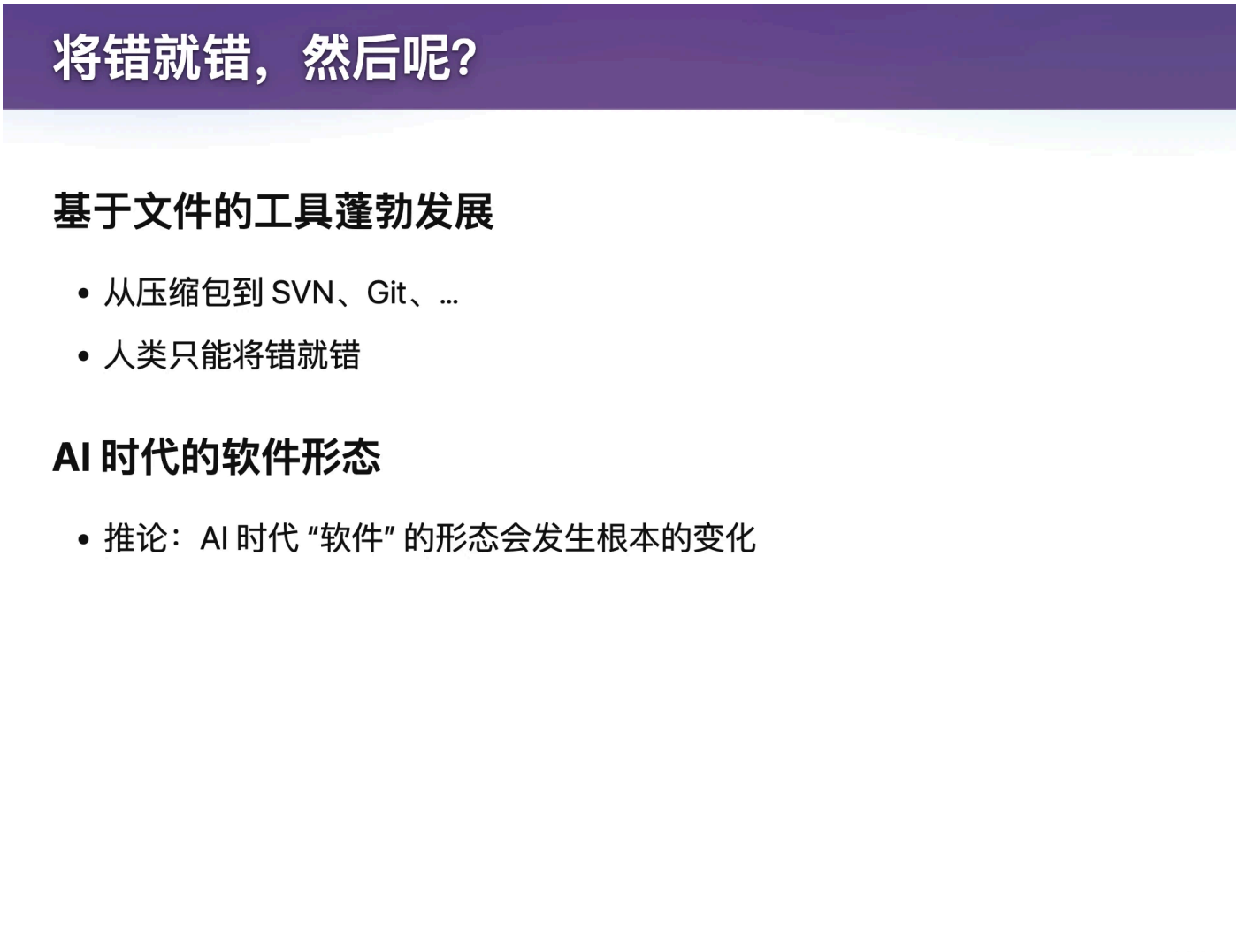
超越文件树的设想：关系图、局部视图与一致性

> **关键判断 · 整理者提炼**
> 结构化仓库设想通过关系与局部视图连接修改沟通和全局一致性，但课堂尚未验证这种保证。

压缩包互传越来越难维护，才需要 SVN、Git 等工具。讲师进一步设想：如果大部分代码由 AI 维护，软件是否仍需以人手编辑的文件集合为核心？可以将文档切片、实现片段、测试等变成对象，并保存“实现了”“验证了”“依赖于”等**关系**。它们自然构成可带环的图，单纯的目录树难以表达这些交叉关系。

黑板上用 **repo.db** 表示这种结构化仓库。它不是课堂已经交付的产品，而是设计推演：让 AI 理解并维护产物之间的一致性，例如文档说链表、实现就应符合；文档要求 A/B/C 三类测试，测试集合就应对应。关联应成为系统可操作的信息，而非只留在人脑与聊天中。

> **整理者说明**
> “AI 理解结构化仓库并自动保证全部一致性”是讲师的未来设想，课堂没有实现或验证这种保证。文件也能承载图的数据；批评针对默认组织和交互方式，不是数学上说文件无法编码图。

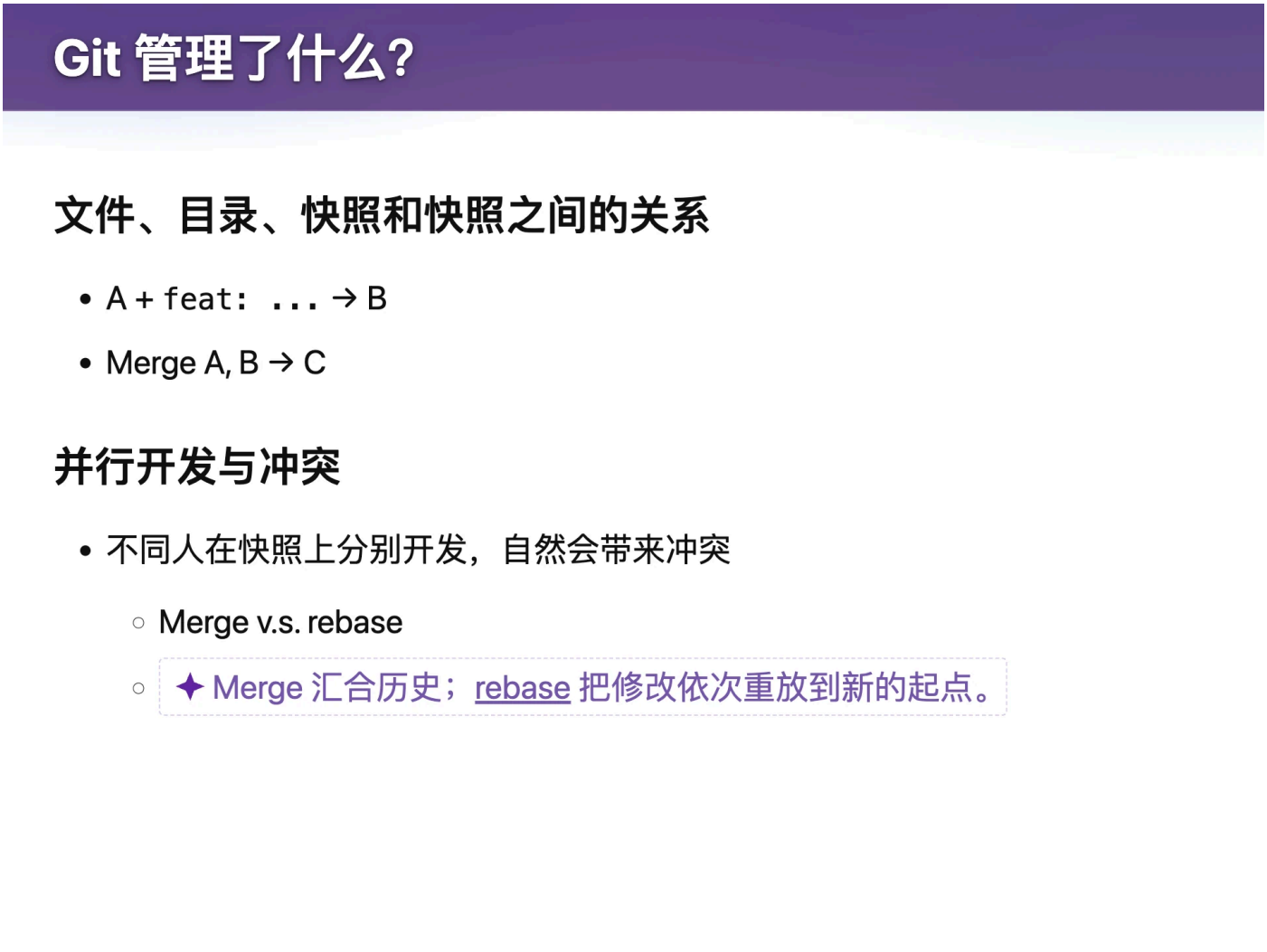


4 / 25

资料图 · 第 4 页 工具演化与 AI 时代软件形态：本页结论是推论。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 4 · 原始讲义 · 工具演化与 AI 时代软件形态：本页结论是推论。

仍然保留 Git 的重要思想：小步前进、版本历史、独立探索后合并。一次修 bug、调整幻灯片样式或新增测试，通常只涉及大型系统的很小部分。Agent 可依据用户当前意图提取相关对象，动态生成可视化看板或界面，让用户只处理这一小步需要的信息。



资料图 · 第 5 页 Git 保存文件、目录、快照和快照关系；merge 汇合历史，rebase 在新起点重放修改。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 5 · 原始讲义 · Git 保存文件、目录、快照和快照关系；merge 汇合历史，rebase 在新起点重放修改。

这与数据库 **View** 的直觉相近：底层资料不变，按当前查询显示相关内容。用户可能不再逐个阅读代码 diff，而是在专门生成的视图中确认修改；系统负责同步受影响的关联产物，并留下从 V0 到 V1 的完整变化。目标是把**局部沟通与全局一致性**连起来。

这个设想仍使用经典计算机科学知识：数据库的实体、**关系**和视图，以及 Git 的版本、小步修改和合并。讲师据此回应“既然 AI 会写，为什么还学基础课”：基础知识支持人重新构想工具。未来是否真能一步完成大型项目尚未知，不能把一两年的课堂猜想当作产品时间表。

第 04 节

00:19:15—00:22:53 · 课堂时间

从“老师教什么”走向自己的理解

> **关键判断 · 整理者提炼**
>
> 真实贡献场景提供学习工具的动机，主动追问比只围绕考试记忆更能形成理解。

学生留言指出：个人作业很少与别人协作，自然不常用 merge、rebase。讲师认可这个场景判断，但提醒大学学习不能只停留在“老师没教、考试不考就不用会”。走进真实开发社区，看 CONTRIBUTING.md，就会碰到分支、提交、PR 与**线性历史**等约定，并由具体需求理解工具。



资料图 · 第 6 页 学生留言引出的学习方法：接触社区规范，形成自己的理解。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 6 · 原始讲义 · 学生留言引出的学习方法：接触社区规范，形成自己的理解。

比如贡献指南要求在个人分支 rebase、避免引入不必要的合并分叉，这时就有了学习 rebase 的**真实动机**。重点不是背命令清单，而是主动留意自己还不理解的概念，再借助经典课程和 Agent 形成自己的**第一性原理解释**。

讲师把 AI 对重复性智力劳动的替代称为“斩杀线”，用来解释学习焦虑与持续更新的必要性。课堂没有提供职业替代的统计模型，其会计从业人数也明确是回忆且不确定，故不作为统计事实记录。

第 05 节 00:22:53—00:28:50 · 课堂时间

Merge 与 rebase：快照身份和修改意图的错位

> **关键判断 · 整理者提炼**

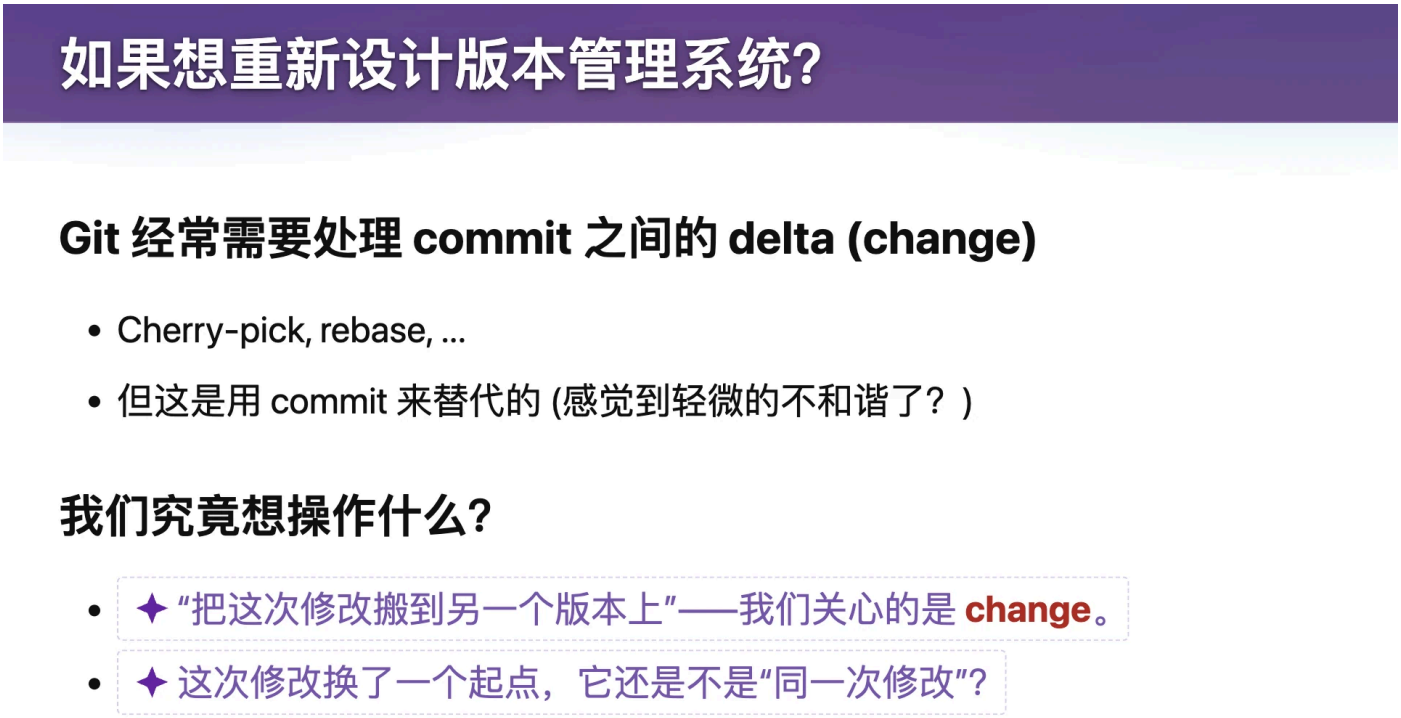
> 换起点后仍想追踪同一次工作，是快照身份与修改意图错位引出的设计问题。

两位开发者从同一快照分别工作。**Merge** 可以创建带两个父提交的新提交，把两条历史汇合起来；原提交仍留在图中。课堂关注的麻烦是冲突解决：最终解决方案是在合并过程中形成的，需要额外理解哪些代码来自谁、哪些决定属于解决冲突的人。

> **整理者说明**
> 整理者校正：merge 不会普遍把 B 的既有提交作者改成 A；官方 git-merge 文档保留两支历史并新增合并提交。课堂的归属问题应限定于合并与冲突解决形成的内容，不应理解成原作者元数据被抹去。

Rebase 则把一条支线上的修改按顺序重放到新的起点。板书例子中，一方先改文档再加测试，另一方先加功能再加测试。把后一方的功能搬到前一方的新状态，再搬测试，可能每一步都遇到内容或语义上的冲突；即使文本能合，也不能推出新组合的测试必过。

每次重放通常创建新的 **commit ID**，因为父提交和快照上下文变了。人想表达的是“把这次功能修改搬过去”，工具却经常用一个 commit 及其与父提交的差异来间接表示修改。由此引出设计问题：能不能让同一次修改在换起点后仍有**稳定身份**？



资料图 · 第 7 页 我们真正想操作的是 change；Git 常通过 commit 表示一次修改。

> **整理者说明**
>
> 课堂对 Git 最初设计倾向的评价是讲师解释；本笔记没有据此断言 rebase 普遍优于 merge。二者解决不同的历史组织需要。

第 06 节

00:28:50—00:33:59 · 课堂时间

Jujutsu：让修改和未完成状态有明确位置

> **关键判断 · 整理者提炼**
>
> jj 用 change ID 追踪工作的演化，并把冲突和未完成的工作副本纳入可保存状态。

Jujutsu（jj）在与 Git 兼容的使用路径上引入 **change ID**。可以把它理解为一项修改在演化过程中的身份：改描述、改内容或 rebase 后，**commit ID** 会变，change ID 通常保留。这样更容易追踪“同一项工作现在变成什么样”，而不只盯某个固定快照。

> **整理者说明**
>
> 官方文档复核：课堂说“change 是一个 object”是教学抽象。jj glossary 的精确定义是：change 表示 commit 随时间的演化，数据模型中没有独立 change 对象，只有 commit 的 **change ID** 属性。

把 Change 作为 First-Class Citizen

祝贺，你发明了 Jujutsu—a version control system

- Rebase 不应该让同一次 change 失去身份。

✦ **Commit ID 会变，change ID 延续**：同一次修改，可以有不同的修订版

- 本。设计说明



资料图 · 第 8 页 change ID 通常延续，commit ID 随重写改变。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 8 · 原始讲义 · change ID 通常延续，commit ID 随重写改变。

另一个设计是把**冲突**作为可以保存的状态。Rebase 后的提交可以携带冲突信息，先完成历史变换，再逐步解决；它没有消灭冲突，而是让未解决的工作仍可被版本系统管理。把“不完整”纳入状态模型，比强迫每一步都假装已经完成更自然。

讲师用频繁被插入紧急工作的情境说明**工作副本**：代码改到一半、不能编译，忽然要修急 bug，Git 可以 stash 再切换；层层打断时要管理一叠未完成工作。jj 的工作副本本身对应一个提交，通常在执行 jj 命令时自动记录变化并更新当前工作副本提交，不必把暂存区当作核心操作步骤。

> 整理者说明
>
> 官方 working-copy 文档说明，**多数 jj 命令**执行时才 snapshot **工作副本**；课堂“每改一个字节立即 commit+amend”应理解为类比，不是逐字节后台提交机制。重写后旧提交隐藏但仍可按 ID 访问，也不是立即物理抹除。

---

jj 官方 glossary 与 working-copy 文档相关段落，2026-10-02读取

官方文档校注 | jj 的精确定义

glossary说明change是commit随时间演化的概念，只有**change ID**是commit属性。working-copy说明多数jj命令会把变化写成**工作副本**的新提交并替换当前修订。正文保留课堂直觉，同时校正“独立change对象”“每字节即时提交”“旧提交立刻抹除”等容易误读的

表达。

原始资料 ↗



资料图 · 第 9 页 工作副本与冲突：把未完成工作保存为可继续操作的状态。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 9 · 原始讲义 · 工作副本与冲突：把未完成工作保存为可继续操作的状态。

> **整理者说明**
>
> 记录暂存状态与发布给他人是不同动作；课堂没有要求把无法构建的中间态发布到公共主分支。

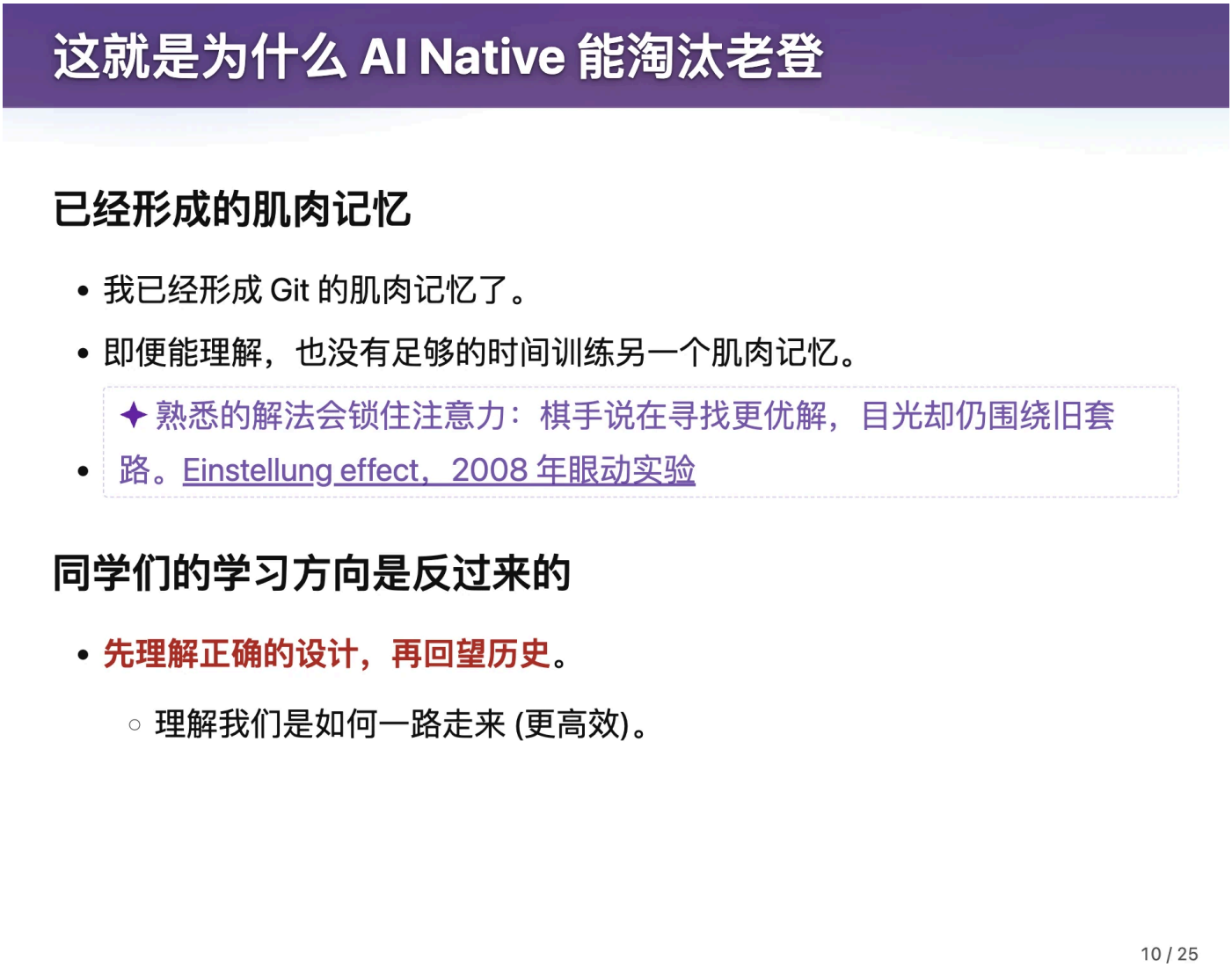
第 07 节 00:33:59—00:40:59 · 课堂时间

肌肉记忆与面向人的工具假设

> **关键判断 · 整理者提炼**
>
> 工具的默认安排受人的工作节奏影响，Agent 改变速度和并行度后需要重新检查这些假设。

理解一个更好的设计，不代表马上能改掉旧习惯。讲师承认自己已经形成 Git 的**肌肉记忆**，即使理解 jj，也没有足够时间重新训练。初学者可以先接触更清楚的概念模型，再回看历史工具为什么那样设计；这比把熟悉误当成唯一合理更有价值。

> **整理者说明**
> “更正确的设计”是课堂评价，实际选型仍取决于团队与工具生态；本课不是要求所有读者立即迁移 jj。



资料图 · 第 10 页 肌肉记忆会限制注意力；先理解设计，再理解历史。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 10 · 原始讲义 · 肌肉记忆会限制注意力；先理解设计，再理解历史。

但生成式软件工程还要再问一步：这些工具**默认使用者**是谁？传统工具主要服务人。讲师把人类类比为单线程个体，手、眼、嘴形成有限的输入输出；开发一个修改可能需要小时，而事先沟通几句或开一次会，就能在编码前划分工作范围。

课堂用“先看位置，再闭眼投粉笔”的比喻解释并发风险：读到的状态到实际写入时可能已经变了，中间其他线程可能运行。这是对**陈旧读取**的教学类比，不能把真实人的感官行为等同于 CPU 指令语义。

人在未完成任务间切换可用 stash、不同尝试可用分支，也能拒绝更低优先级工作，使积压保持可管理。Agent 的速度与**并行度**改变之后，这些在人类工作节奏下合理的默认安排，需要重新检查。



资料图 · 第 11 页 人类的单线程类比与 stash、branch 的使用场景。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 11 · 原始讲义 · 人类的单线程类比与 stash、branch 的使用场景。

第 08 节

00:40:59—00:46:58 · 课堂时间

多 Agent 演示：生成加速以后，协调也必须加速

> **关键判断 · 整理者提炼**
>
> 多 Agent 扩大产量的同时也放大干扰，隔离、验证与整合必须一起改进。

单个 Agent 可以在分钟甚至秒级完成一次改动，再把并行数量放大十倍、百倍，传统先开会划分任务、各自慢慢做的节奏就会失衡。测试流水线可能限制提交速度，但不能自动解决并发读写与协调问题。生成更快，意味着**协调、验证和整合**也要更快。

但如果是 Agent Team (Swarm) 呢?

单个 Agent 快，Agent 还可以并行 😂

- 单个 Agent 工作的速度就显著比人更快：分钟级产生一个原子提交
- Agent 还可以并行

Git “为人设计” 的模型跟不上这个速度了

- ✦ 一个工作目录里，A 正在修改，B 就可能读到一半的结果
- ✦ 现在需要一起加速的，还有协调、验证与整合

资料图 · 第 12 页 Agent 更快且可并行：共享工作目录容易暴露中间态。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 12 · 原始讲义 · Agent 更快且可并行：共享工作目录容易暴露中间态。

一个直接办法是让每个 Agent 像**独立开发者**一样拥有自己的机器或容器，通过远程 Git 仓库协作。课堂用两个 tmux 会话展示主 Agent 派发任务、子 Agent 同时工作；主 Agent 可以观察输出、收取总结，再注入新指令。讲师还举出由一种模型设计、另一种模型复核的配置，这是组织方式的例子，不是质量保证。



视频原帧 46:22 原视频：主 Agent 派发任务、两个会话并行生成大量代码；日志自报不等于独立验证。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 00:46:22 · 46:22 原视频：主 Agent 派发任务、两个会话并行生成大量代码；日志自报不等于独立验证。（右侧屏幕裁切；原始全画幅留存于证据目录。）

实际演示让两边生成大量 slop 代码，很快出现上千行结果。画面说明并行会话与任务派发确实在运行，却不能证明这些生成代码有产品价值或通过了独立质量审计。若多个 Agent 直接写**同一个工作目录**，另一个 Agent 可能读到半成品、覆盖修改或错认仓库状态；**隔离工作空间**首先解决的是这类相互干扰。

第 09 节

00:46:58—00:51:06 · 课堂时间

Git worktree：隔离工作目录，共享仓库对象

> **关键判断 · 整理者提炼**
>
> Worktree 隔离工作文件与部分 Git 状态，共享仓库资料，却仍需整合并验证最终结果。

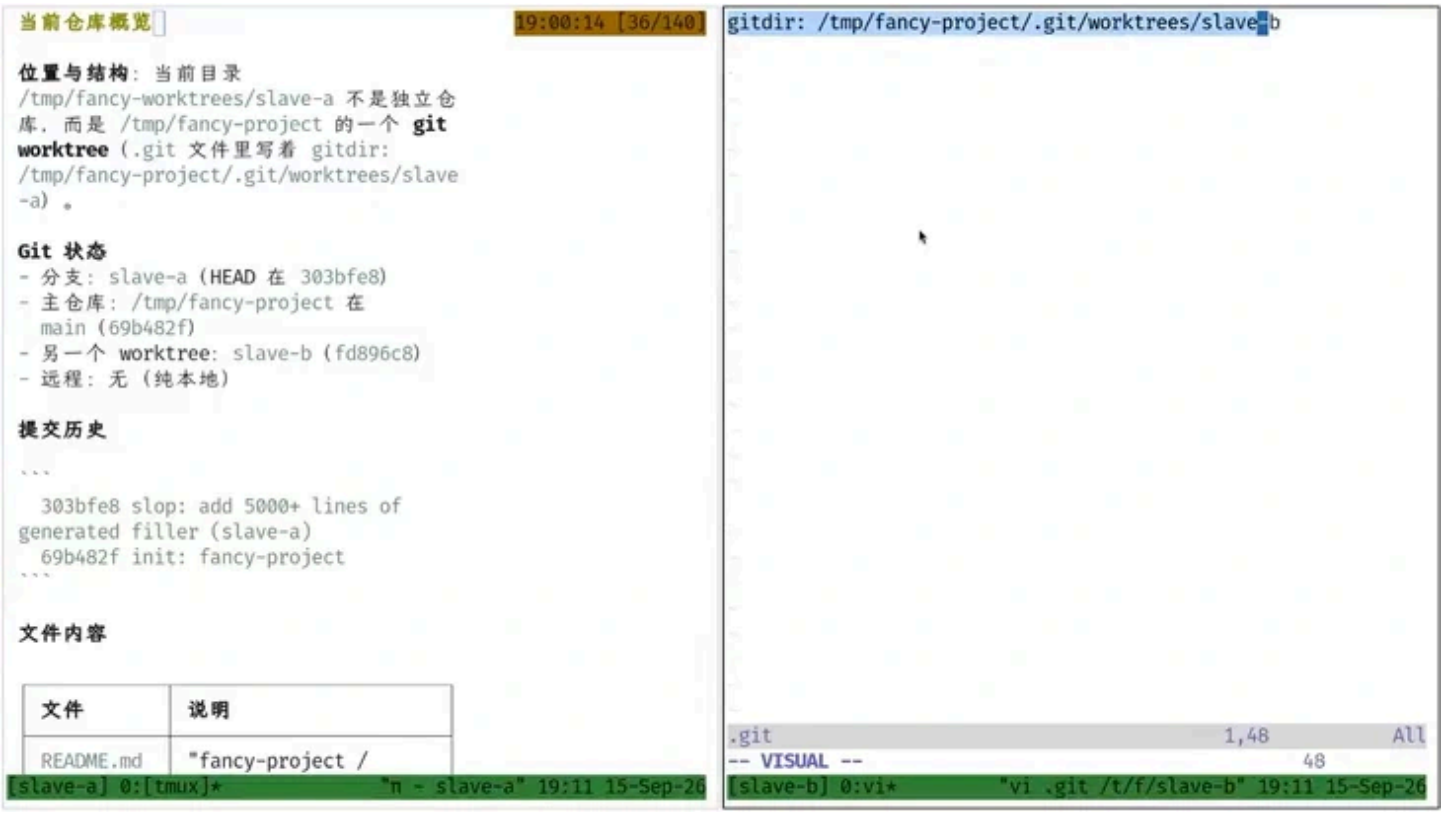
讲师把“让两个 tmux 会话分别工作在 worktree”交给 Agent，随后检查各进程的当前目录、分支和提交历史。Worktree 为同一仓库提供多个工作目录，各自拥有工作文件、**HEAD 和 index**；**共享对象库**与常规分支引用，因此一个工作树中新建的提交可直接在本地被其他工作树看见，不必先 push 再 pull。



视频原帧 48:25 原视频：进程工作目录与独立分支状态。（右侧屏幕裁切；原始全画幅留存于证据目录。）

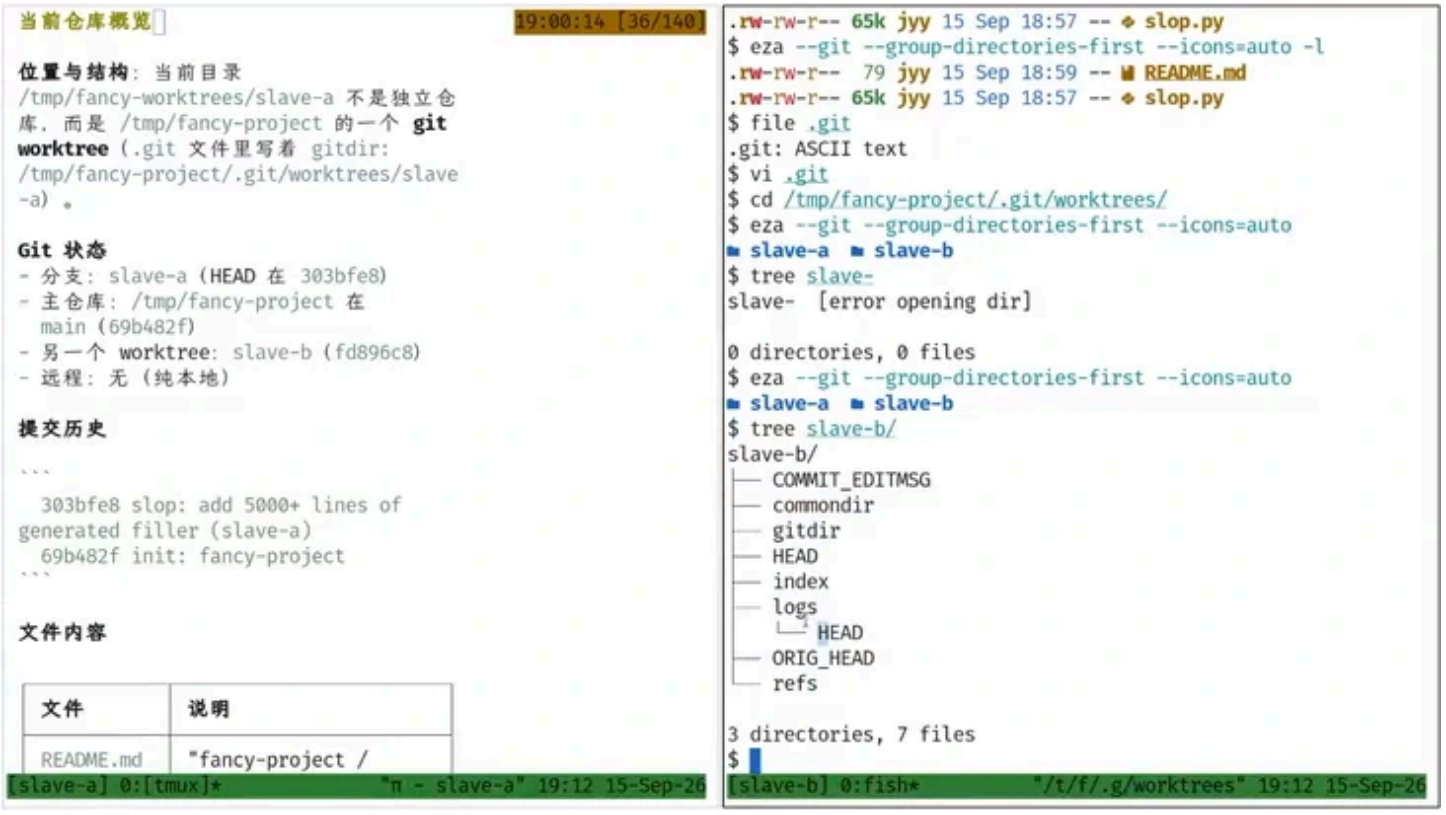
视频原帧 00:48:25 · 48:25 原视频：进程工作目录与独立分支状态。（右侧屏幕裁切；原始全画幅留存于证据目录。）

现场查看 linked worktree 的 .git，发现它不是普通仓库里的目录，而是一个文本文件，内容为 **gitdir:** 加路径，指向主仓库 .git/worktrees 下对应的管理目录。继续查看该目录，可见 HEAD、index、logs、commondir 等管理信息。文件系统中的间接引用，使隔离工作目录可以复用已有仓库数据。



视频原帧 49:50 原视频：.git 文本中的 gitdir 指向主仓库 worktrees 管理目录。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 00:49:50 · 49:50 原视频：.git 文本中的 gitdir 指向主仓库 worktrees 管理目录。（右侧屏幕裁切；原始全画幅留存于证据目录。）



视频原帧 50:20 原视频：管理目录里的 HEAD、index、logs 等文件。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 00:50:20 · 50:20 原视频：管理目录里的 HEAD、index、logs 等文件。（右侧屏幕裁切；原始全画幅留存于证据目录。）

官方讲义给出的示例是 git worktree add -b agent-a ../gense-agent-a，另一个 Agent 用 agent-b 和另一目录。命令表达的原则是“每人自己的分支与工作目录”，不是复制一整套远程协作系统。彼此代码最终仍须 merge 或 rebase，隔离不会自动化解**逻辑冲**

突。

> **整理者说明**
> Git 官方 worktree 手册复核了每工作树独立 HEAD/index、共享其他仓库资料的边界。链接工作树是文件与 Git 状态隔离，不等同于进程、网络、凭据或操作系统安全沙箱。

Git 官方 git-worktree/git-merge 手册相关段落，2026-10-02读取

官方文档校注 | worktree 与合并边界

Git官方手册说明linked worktree共享仓库资料，但HEAD、index等为各工作树独立；refs也有特例。Merge保留被合并的历史，并在非快进合并时新增合并提交，不能把冲突解决者的责任问题表述成原作者普遍被改写。

原始资料 ↗



资料图 · 第 13 页 一个仓库、多个工作目录；共享对象不代表自动合并。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 13 · 原始讲义 · 一个仓库、多个工作目录；共享对象不代表自动合并。

第 10 节

00:51:06—00:55:32 · 课堂时间

历史记录的价值：从归责转向恢复和教训

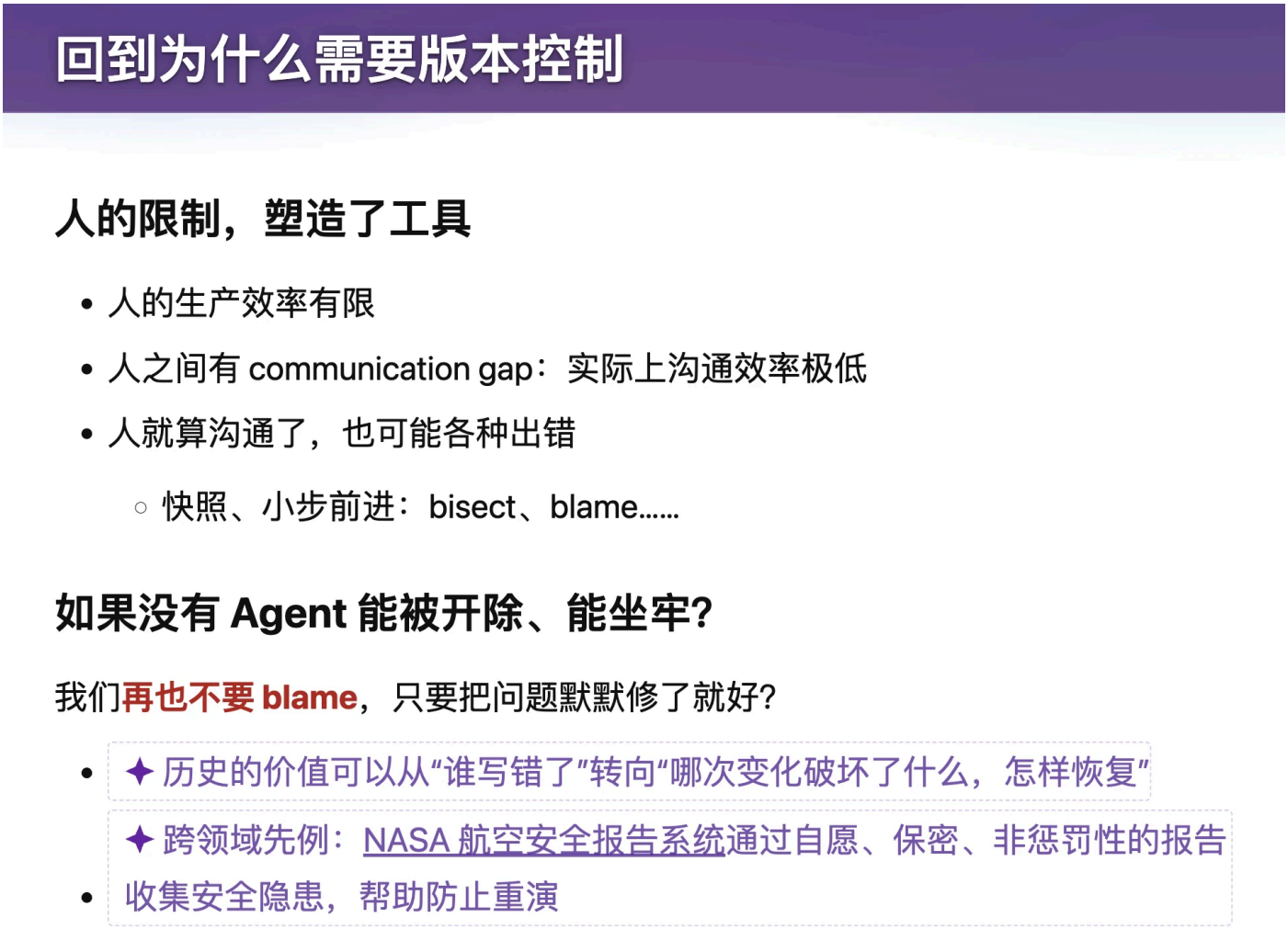
> **关键判断 · 整理者提炼**
>
> 历史既能定位回归，也能留下可恢复快照与失败教训，不能把课堂修辞理解为不再追查错误。

人的产能有限、沟通有损，即使每天同步进度，理解仍可能错位。**小步提交**、快照、bisect 和 blame 使团队可以定位某次改变，再回忆当时的需求和沟通背景。规模扩大后，沟通成本会吞掉部分新增人力，人数十倍并不保证产出十倍；课堂的两三倍举例不是经测量的普遍定律。

讲师提出挑衅性问题：Agent 不能像员工一样被开除或承担惩罚，版本系统还要围绕“谁写错了”组织吗？他设想更侧重直接推进项目和修好当前错误，让历史留下演化过程、**可回退快照**与 lessons。

这段口述一度走得更远，说可能不必追查哪次提交引入错误；随后又明确承认保留快照的必要性，因为走错方向后回到早期状态可能比继续修更便宜。笔记保留这两层，不把修辞变成“以后无需定位回归”的实践禁令。历史仍可帮助解释错误、**重现问题**与避免再次走错。

> **整理者说明**
> 讲义将历史价值表述为“哪次变化破坏了什么，怎样恢复”，比单纯取消 blame 更准确。官方页中的 NASA 非惩罚安全报告只是跨领域类比，本次未独立审阅其制度材料。



资料图 · 第 14 页 人的限制塑造工具；版本历史可以服务定位、恢复与教训。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 14 · 原始讲义 · 人的限制塑造工具；版本历史可以服务定位、恢复与教训。

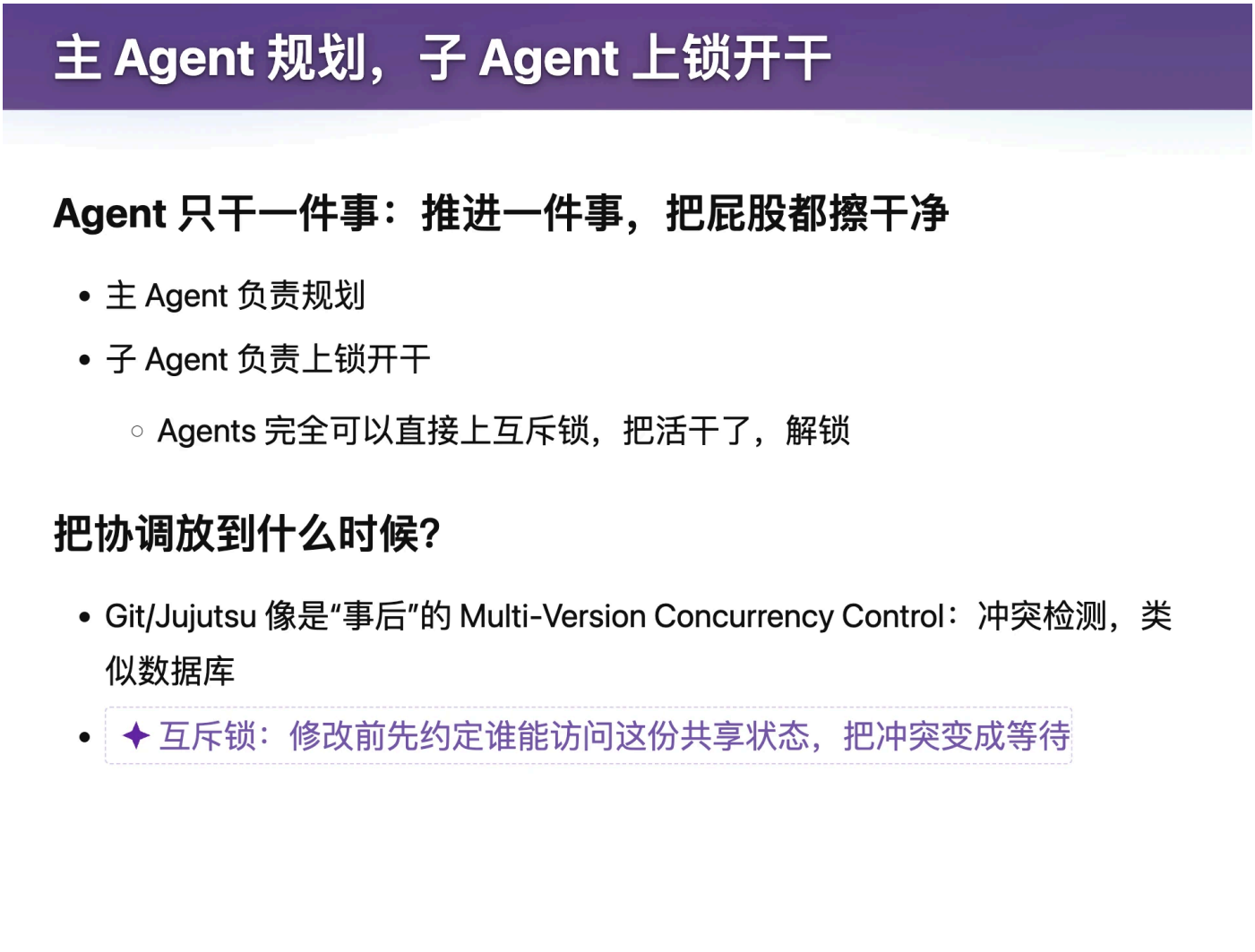
> **整理者说明**
> 同一模型的多个 Agent 并不保证判断一致或正确；课堂关于克隆体趋同的说法不能作为省略复核的理由。

合并冲突还是提前上锁：协调发生在何时

> **关键判断 · 整理者提炼**
> 提前协调是否划算取决于依赖范围与冲突成本，锁还必须配合最新读取、验证和失败恢复。

讲师将 Git 的分支后合并类比为**乐观并发控制**：先假定各方修改不会冲突，各自推进，最后检测并处理不兼容。数据库事务中，也可能先读取、计算再更新，期间产生数据依赖、冲突或死锁，需要中止或回滚。二者用于比较**协调时机**，不意味着 Git 提供数据库事务的全部语义。

在传统团队已经沟通范围、每次修改较慢的情况下，先独立做再整合通常有效。Agent 更快、任务更多，主 Agent 若一次派出大量相互关联工作，可能频繁付出冲突修复成本。另一种方案是先声明对共享资源的使用，再上锁、工作、提交、解锁；后来者等待锁释放。



15 / 25

资料图 · 第 15 页 主 Agent 规划、子 Agent 上锁干活：把部分冲突转成等待。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 15 · 原始讲义 · 主 Agent 规划、子 Agent 上锁干活：把部分冲突转成等待。

锁把对同一资源的并行访问变为有序访问，因此本质上会牺牲该范围的**并发性**。不过若系统还有许多互不冲突的任务，其他 Agent 仍可工作。关键不是“锁一定优于合并”，而是资源划分、冲突概率和验证成本是否使提前协调更划算。

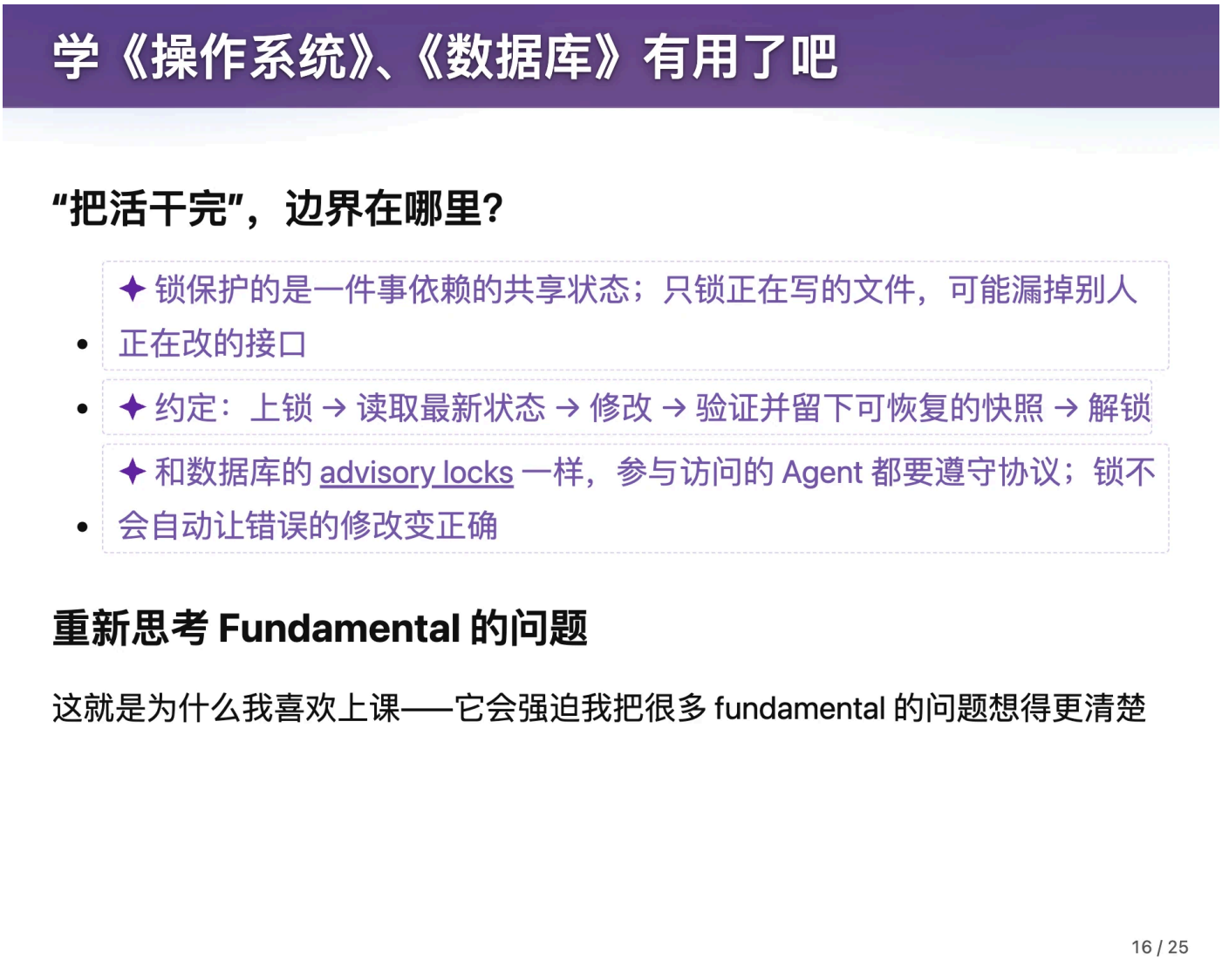
> **整理者说明**
> 官方讲义补全了锁的责任边界：保护任务依赖的**共享状态**，不能只锁正在写的文件而漏掉别人正改的接口。流程应是上锁→读最新状态→修改→验证并留可恢复快照→解锁；所有参与者须遵守协议。锁不会自动让错误实现变正确。

官方课程讲义，课堂未逐项展开

讲义补充 | 锁也需要恢复与失败策略

只规定上锁/解锁不够：Agent崩溃后谁释放锁，多个资源如何避免循环等待，读取发生在持锁之前还是之后，都需要约定。大范围锁减少冲突却压缩并行；细粒度锁提高并行却增加依赖管理。依靠所有参与者共同遵守的advisory协议，本身不是正确性证明。

PDF物理页 15、16 · 原始资料 ↗



16 / 25

资料图 · 第 16 页 锁的边界、最新状态、验证与可恢复快照。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 16 · 原始讲义 · 锁的边界、最新状态、验证与可恢复快照。

> **整理者说明**
> 课堂用学生各自查看或修改个人数据说明低冲突情境；整理者提醒，共享课程容量、公共配额等操作仍会争用**共享状态**，不能外推成教务数据库天然没有冲突。

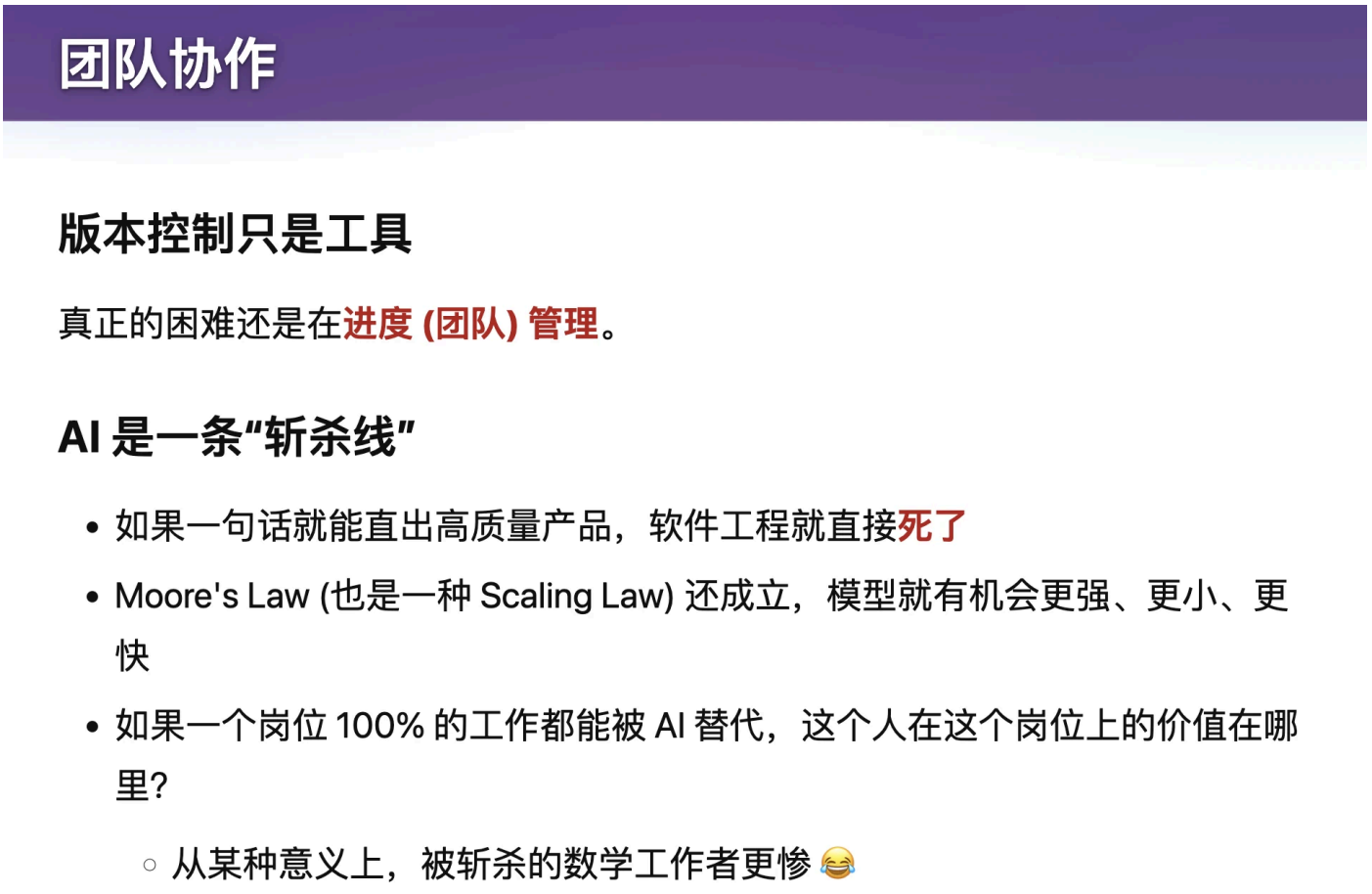
第 12 节

01:03:56—01:08:29 · 课堂时间

工具之外：任务分解、团队边界与整合策略

> **关键判断 · 整理者提炼**
> 管理并行工作的关键是划清可独立推进的边界，再对组合后的产品负责。

版本控制只是工具，真正困难的是团队进度与**协调**。软件工程仍有价值，是因为一句模糊想法还不能稳定变成高质量的大型产品。把“如果一句话能生成完整产品，某类工程工作就失去价值”看成条件命题，可以理解课堂焦虑；它不是已经实现的能力，也没有可靠日期。

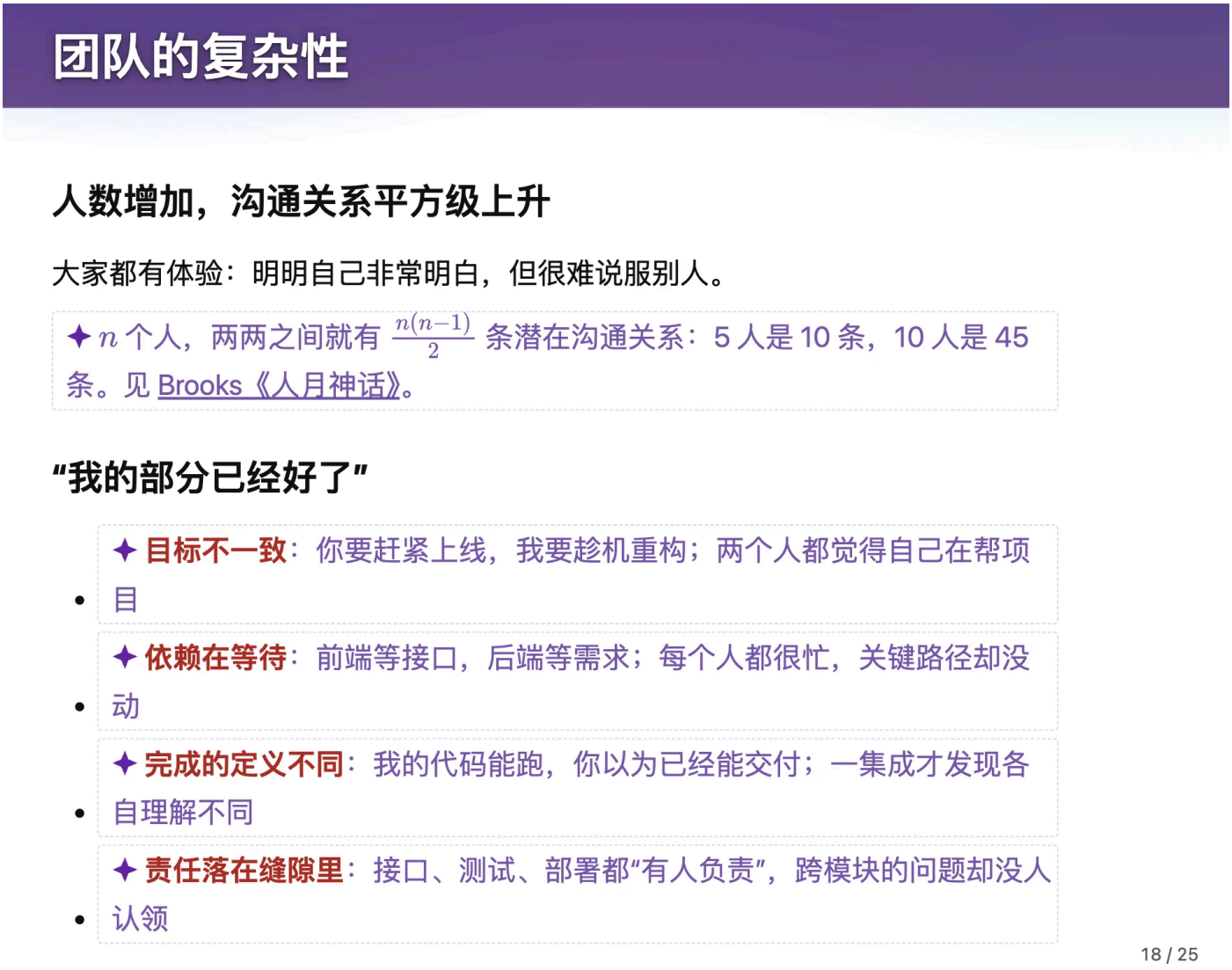


资料图 · 第 17 页 版本控制只是工具；团队管理仍需任务分解与协调。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 17 · 原始讲义 · 版本控制只是工具；团队管理仍需任务分解与协调。

高价值的管理工作是将整体产品分成可独立推进的部分，同时保证最后能拼回整体。新增一个参与者会增加潜在**沟通关系**；大家都把自己那份做好，仍可能因为不理解别人正在改什么而在接口或测试上相撞。

> **整理者说明**
> 讲义补充：n 人两两潜在关系为 n(n−1)/2，5人10条、10人45条。这是关系计数，不是沟通时间必然按此比例增长的经验定律。典型摩擦包括目标不一致、依赖等待、对“完成”的定义不同，以及跨模块责任无人认领。



资料图 · 第 18 页 沟通关系与四类团队摩擦。

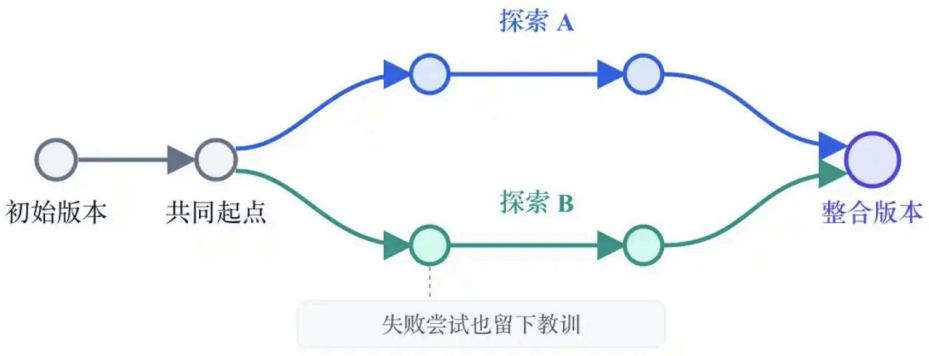
官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 18 · 原始讲义 · 沟通关系与四类团队摩擦。

现在就能用两三个 Agent 练习这项能力：如何分任务、怎样**定义边界**、何时提交、何时合并、由谁验证整合结果。工作流要利用版本工具保留**可恢复状态**，也要让主 Agent 能判断整个产品是否在推进。学习的目标是形成软件工程直觉，不是单纯增加子 Agent 数量。

> **整理者说明**
> 讲义图还提醒保留**失败尝试**与教训，再把并行探索整合为新版本。Agent Memory 可以承接部分记忆责任，但并非自动可靠。

版本控制留给我们的启发

项目可以不仅是“当前状态”



记住历史 · 并行探索 · 汇合成果

技术会过时，First Principles 会留下

能够看到开发的历史、记住教训 (也许会被 Agent Memory 替代)，也能管理多个并行探索，并最终合并。

19 / 25

资料图 · 第 19 页 记住历史、并行探索、汇合成果；失败尝试也留下教训。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 19 · 原始讲义 · 记住历史、并行探索、汇合成果；失败尝试也留下教训。

第 13 节

01:08:29—01:13:04 · 课堂时间

单向汉诺塔：用执行轨迹解释程序

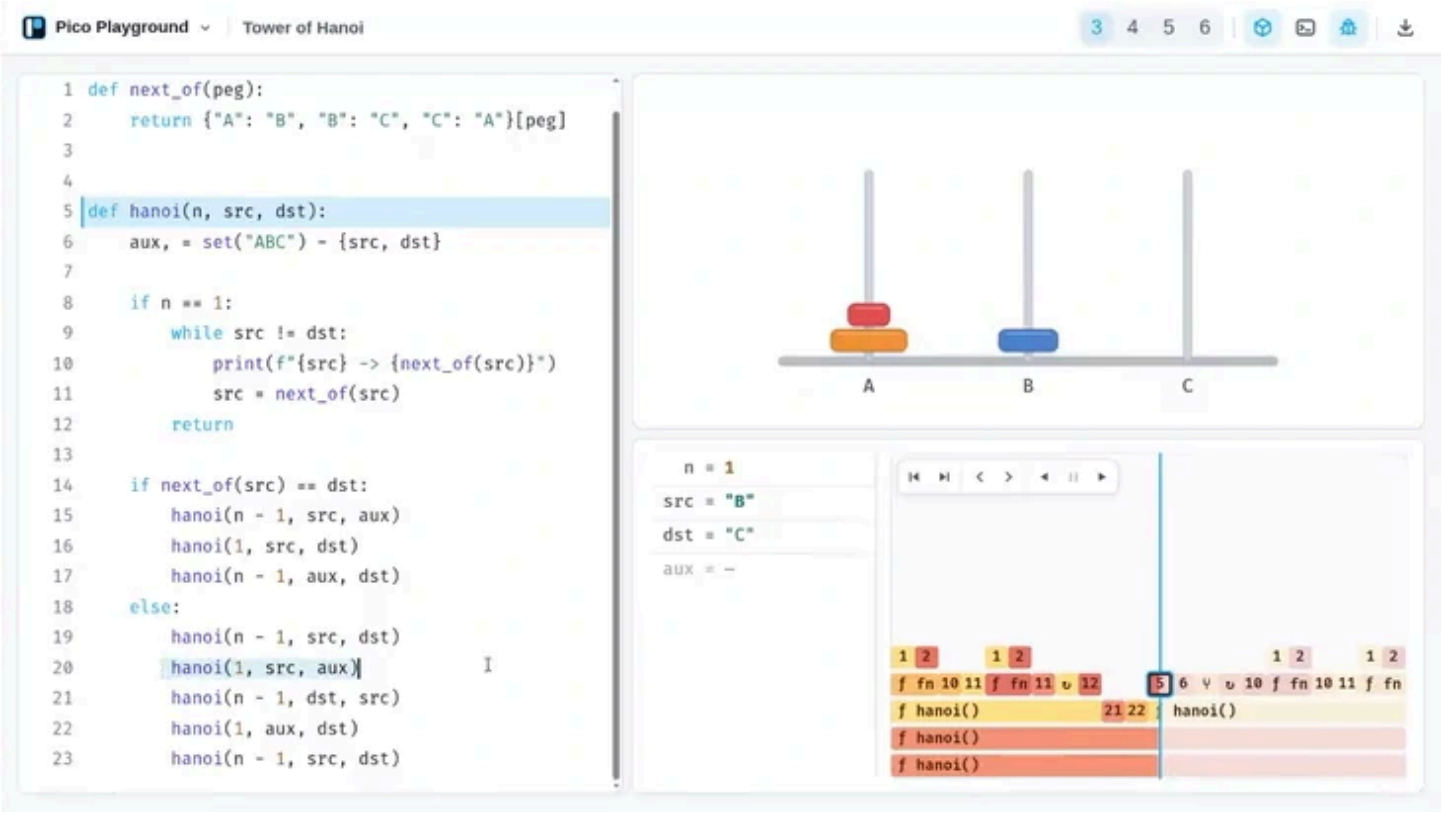
> **关键判断 · 整理者提炼**
>
> 额外的移动方向约束改变递归结构，执行轨迹帮助定位错误，有限验证不替代一般证明。

讲师拿出一个额外约束的汉诺塔题：盘子仍须遵守小盘在大盘之上，但移动方向只能沿 A→B→C→A 环行。因此不能把普通汉诺塔算法里的 A→C 直接搬盘照抄过来；若目标不是当前柱的下一柱，最大的盘子必须经过中间柱。

> **整理者说明**
>
> 与第06课的普通非递归汉诺塔区分：这里的额外条件是单向移动。不能直接套用普通汉诺塔的 2^n−1 作为本题步数。

他用最大盘子的位置来组织递归：先把上层盘子挪开，让最大盘子向前一步，再为第二步安排空间，最后把小盘重新叠到目标。这体现了围绕**关键约束**分解问题的方法。课堂用归纳思路解释最优性，本笔记没有仅凭演示画面宣称完成完整最优性证明。

Pico Playground 同时显示源码、三根柱的状态、当前变量与调用轨迹。修改代码后可视化跟着执行变化；出现非法搬盘时，可以定位是哪条语句产生了错误。价值不仅是看最终答案，而是让执行过程**可观察**、**可回放**，帮助学生找到自己在哪里想错。



视频原帧 72:00 原视频：单向汉诺塔源码、塔状态和调用轨迹；移动方向为 A→B→C→A。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:12:00 · 72:00 原视频：单向汉诺塔源码、塔状态和调用轨迹；移动方向为 A→B→C→A。（右侧屏幕裁切；原始全画幅留存于证据目录。）

> 案例回扣 整理者复核屏幕递归：n=3 从 A 到 B 需15步，到 C 需21步，均不同于普通汉诺塔的7步。对 n=1..7、目标B/C逐次检查单向移动和大小盘合法性，并用独立广度优先搜索比较最短步数，14组全部一致；这是**有限规模验证**，不替代一般性最优证明。

> 整理者说明
>
> 口述转写对某次错误移动的大小盘方向有混乱；笔记按汉诺塔规则描述为非法搬盘，不把含混口述写成相反规则。

第 14 节 01:13:04—01:18:05 · 课堂时间

从教学工具到信息系统：先认清需要保存什么

> 关键判断 · 整理者提炼
>
> 教学产品的核心若是探索历史，应先让存储表达版本与点评关系，再决定数据库方案。

假设把 Playground 做成产品，增值服务可以是学生卡住时调用 AI 助教。助教需要知道学生此前写过什么、怎样尝试、在哪里出错，再给个性化指导；因此产品从一次性的运行环境变成需要账户、记录、点评与历史的信息系统。课堂的定价和商业化只是设想，没有用户或收入验证。

模型给出的常规方案是 API 应用、关系数据库和对象存储；需要异步点评时加后台任务，任务量小时数据库任务表即可。讲师不喜欢直接落到通用信息系统模板，因为这里最核心的需求是**版本**与**探索过程**，而不是先选数据库栈。



20 / 25

资料图 · 第 20 页 常规模型方案：API、数据库、对象存储与按需后台任务。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 20 · 原始讲义 · 常规模型方案：API、数据库、对象存储与按需后台任务。

他以 Overleaf 可导出为 Git 仓库的使用经验作类比：一旦内容能够被现有**版本**工具管理，熟悉的 Agent 和协作流程就能直接利用。对于学生代码，可以在运行、暂停或切换等有意义的节点保存版本，把 **AI 点评**关联到当时版本，并保留分叉、重试和 diff。

> **整理者说明**
>
> 官方讲义补充：**git notes** 可以后补点评而不改原提交；保存汉诺塔动作序列比只保存最终盘面更适合回放与比较。哪一步有教学意义仍需应用判断，Git 只提供历史与差异。



资料图 · 第 21 页 把探索过程、版本差异与点评关联起来。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 21 · 原始讲义 · 把探索过程、版本差异与点评关联起来。

早期甚至可以先在已有代码仓库中探索这些数据和提交约定，待目录格式、记录方式和使用流程收敛，再决定是否迁移到数据库。目的不是宣布 Git 比数据库普遍更好，而是让存储表达真实问题，避免过早为不重要的压缩或性能指标优化。

> **整理者说明**
>
> 用户关系、权限、检索统计等数据仍可能适合**关系数据库**；**版本**轨迹与业务实体可各用适合的表示。

第 15 节　　01:18:05—01:27:05 · 课堂时间

现场从零生成：满足文字，不等于满足意图

> **关键判断 · 整理者提炼**
>
> 模型逐项实现需求清单，仍可能偏离真正的产品需要，早期对齐不能靠增加子 Agent 数量替代。

讲师从课程作业要求现场启动生成，事先没有准备结果。他建议项目最早期先与一个能力足够的 Agent 多轮讨论，自己承担产品与方向判断；此时**意图尚未对齐**，直接把任务摊给许多子 Agent 并不能替代思考。课堂同时比较直接粘贴要求开干与先沟通的思路。

模型倾向认真实现提示里列出的每一项，但讨论稿中的每个细节未必同等重要。这个作业真正的中心是“持续为你工作的个人助手”。对于学生，最近一个月最痛的可能是学习与复习：收集外部资料、分层归档、综合总结、根据学习进度提供合适内容和好用界面。

讲师追问生成项目能否满足学习助手需要，发现它更多在搭数据库、邮件回复和 eHall 业务结构。每个模块看起来都符合清单，整体却未必是用户最想要的产品。这个例子具体体现 **Intent–Spec Gap**：文字被忠实实现，并不能消除文字与**真实需要**之间的偏差。



资料图 · 第 22 页 从零开始连需求也未对齐；确定得越早，越可能把误解固定下来。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 22 · 原始讲义 · 从零开始连需求也未对齐；确定得越早，越可能把误解固定下来。

eHall 也可以只是一种验证场景：讲师本意可能是能利用个人信息在各类网页办事的 **browser-use Agent**，而不是围绕一个教务站点先造复杂 job 系统。生成过程中甚至尚未初始化 Git 仓库，说明项目管理策略也不能假设模型会自行补齐。

> **整理者说明**
> 这里评价的是当堂具体输出，不能推出所有模型或所有无人值守生成都只会产生 slop；讲师关于不同模型可能初始化仓库的比较未形成受控测试。

第 16 节

01:27:05—01:30:48 · 课堂时间

小步推进：保留现在、历史和教训

关键判断 · 整理者提炼

小步推进要同时保存现状、演化和失败原因，并用后续任务中的实际采用来检验记忆的作用。

项目开始前应先形成管理模型，再决定下一**小步**走向哪里。成熟项目通常已有架构、接口、测试和惯例作上下文，维护任务更容易受约束；从零开始时这些都没有，第一批决定会把系统带向某个方向。

架构要回答的不只是现在能不能运行，还包括未来需求改变时，是否能以较小的**局部修改**应对。AI 助教以后可能是文字、图像、可交互或非交互，模型与协作方式也会变化。若早期架构把这些变量绑死，后续功能越多，修改越难。

因此版本管理在两个时代都要记住三件事：现在有什么，**曾经怎样演化**，**已经从失败中学到什么**。被删掉的内容可能后来有用，走不通的路线也应留下原因。仅保留最终代码，会失去当时为何那样设计以及哪些办法失败过；下一轮对话便可能从头踩同一个坑。

官方课程讲义，课堂未逐项展开

讲义补充 | “会成长”应能被观察

成长不是只增加日志。可用一个**可复核闭环**衡量：保存一次明确的偏好或纠正，在下一次相关任务中真正采用，并允许再修正。记忆需要区分长期偏好、暂时目标与已经失效的信息；当前状态、历史和教训各有不同作用。

PDF物理页 1、19、23 · 原始资料 ↗



资料图 · 第 23 页 代码保留结果，决策记录保留原因与走不通的路。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 23 · 原始讲义 · 代码保留结果，决策记录保留原因与走不通的路。

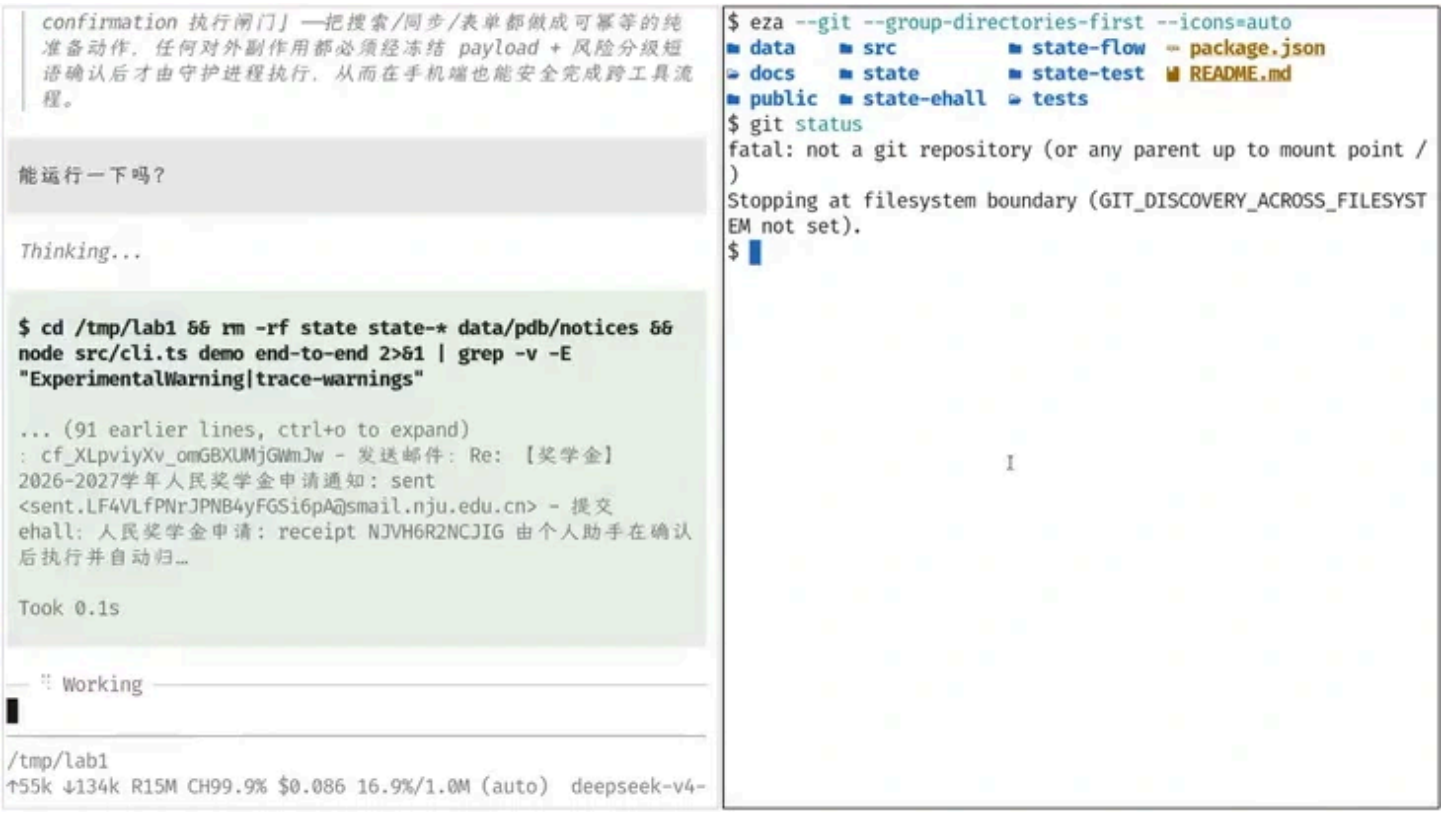
第 17 节

01:30:48—01:32:53 · 课堂时间

助手 demo 的真实边界：流程跑通与外部接通不同

> **关键判断 · 整理者提炼**
>
> demo 流程中的成功日志不等于真实外部执行，需区分真实代码、替身数据与尚未验证的适配器。

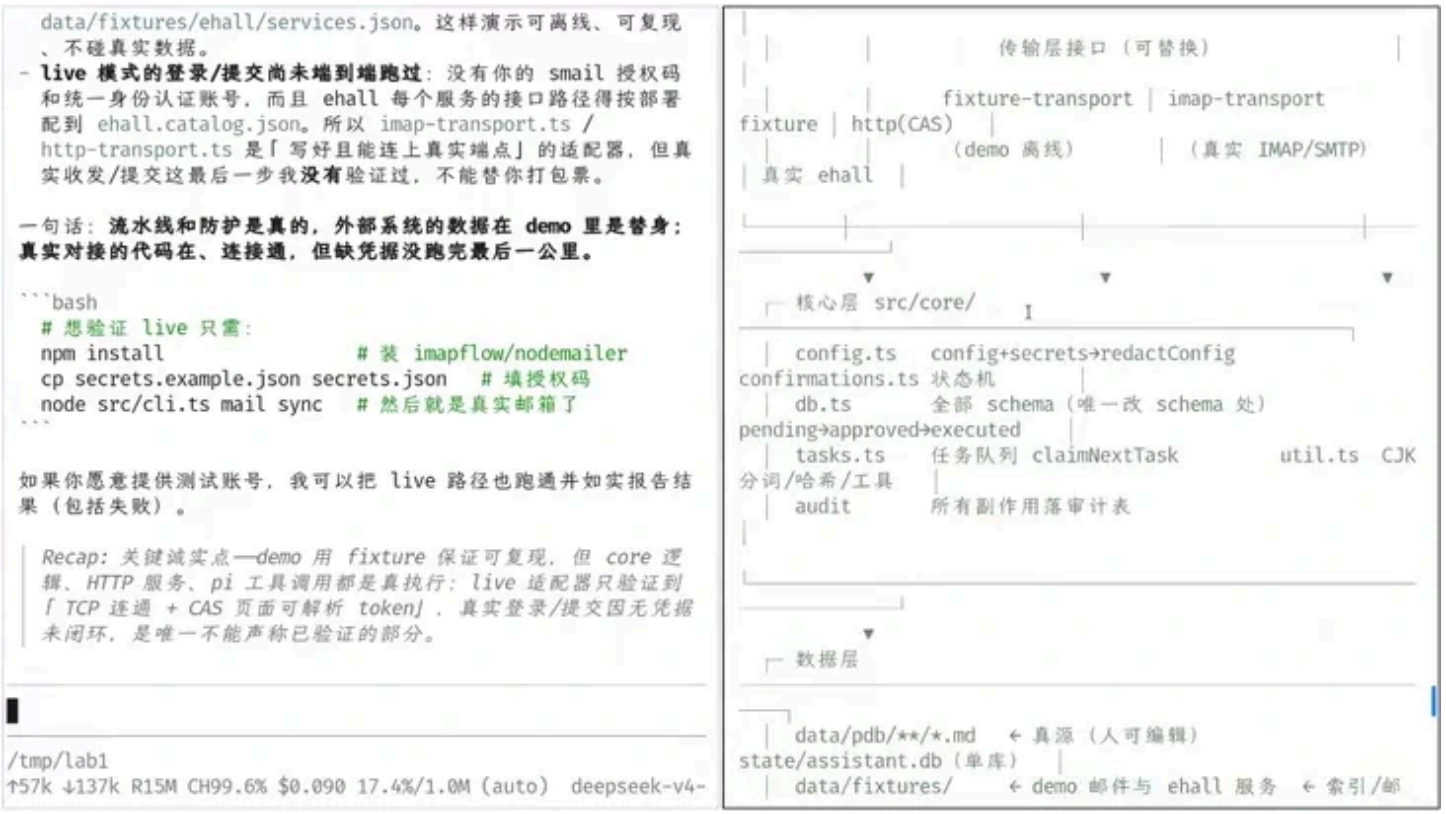
课堂查看刚生成的助手并让它运行。约91:00画面右侧仍可见 git status 报“**not a git repository**”；随后讲师在91:19左右看到它终于做了一个提交。这个前后变化说明：晚些时候补了版本记录，不等于此前的小步演化已经被保存。



视频原帧 91:00 原视频：demo 日志与尚未初始化 Git 的报错；日志中的 sent 不代表真实发信。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:31:00 · 91:00 原视频：demo 日志与尚未初始化 Git 的报错；日志中的 sent 不代表真实发信。（右侧屏幕裁切；原始全画幅留存于证据目录。）

端到端 demo 的日志显示邮件 sent、eHall receipt 等结果。但稍后的模型说明明确区分：流水线与防护逻辑使用真实代码，外部系统数据在 demo 中使用 **fixtures**；尚缺真实邮箱授权和统一身份认证材料，**真实适配器**“最后一公里”还没验证。不能把这段 demo 写成成功发送真实邮件、办完真实 eHall 事务。



视频原帧 92:00 原视频：模型明确说明 demo 用替身数据、live 尚未验证，并展示分层架构。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:32:00 · 92:00 原视频：模型明确说明 demo 用替身数据、live 尚未验证，并展示分层架构。（右侧屏幕裁切；原始全画幅留存于证据目录。）

屏幕中的架构分传输层、编排/能力与核心层、数据层。核心包含**确认状态机**、任务队列、审计和 SQLite；传输侧可替换 fixture、HTTP/CAS、IMAP/SMTP。讲师没有完整 review 实现，只指出它已经像一个业务信息系统，而这可能偏离他最初想要的轻量助手。

> **整理者说明**
>
> 画面补充｜稍后模型展示的设计包括统一 executeConfirmation 入口、pending→approved→executed 状态、确认时冻结收件人/正文等 payload、Message-ID 与 idempotency_key 去重、任务持久化。这些是屏幕可读的模型设计说明；本笔记没有读取该项目源码或独立验证这些防护。

第 18 节

01:32:53—01:35:43 · 课堂时间

MVP：先确认最需要什么，再决定数据结构

> **关键判断 · 整理者提炼**
>
> 先做能验证关键需要的最小可用版本，再根据真实的数据组织问题决定是否引入数据库。

Minimum Viable Product 的作用，是用最小可用版本验证**最重要的假设**。产品最大的风险之一，是做出了没人要的东西；课堂项目已经做出一堆功能，却有很多不是讲师想要的。选择 MVP 会迫使人说清哪些需求绝对要、哪些可以暂缓。

整理者说明

讲义补充：先选最值得验证的假设，再造能验证它的**最小版本**，并明确什么观察结果会支持或推翻假设。MVP 不是按比例缩小完整系统，也不是只做一个好看的演示。



资料图 · 第 24 页 MVP 验证需求：立即可用但不完备，先验证最关键假设。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 24 · 原始讲义 · MVP 验证需求：立即可用但不完备，先验证最关键假设。

若最重要的是提升学习和工作效率，第一个版本可能只需打通资料进入、通知进入、归档与使用流程，未必立刻需要数据库。文件目录是用户熟悉、可直接查看和调整的表达，数据库 schema 则增加一层概念映射与迁移成本。这里应根据需要选择简单表示，而不是把“不用数据库”当作原则。

讲师介绍自己邮件助手的做法：按 email ID 在共享云盘中保存 Agent 解析结果、回复草稿、分类等，再交给下一层流水线处理。先想清楚有哪些阶段、如何协作、什么信息需要跨设备共享，比一开始就锁定通用业务系统架构更重要。

当数据组织真正膨胀，再提炼数据库可能更合适。但这仍需识别数据含义和迁移规则；让 Agent 执行迁移，不会使**数据一致性**与验证责任自动消失。

追加云盘同步：用真实变更检验架构

> **关键判断 · 整理者提炼**
>
> 追加云同步可以检验架构的修改成本，但模型提出的分类与冲突方案还需要实现和多设备验证。

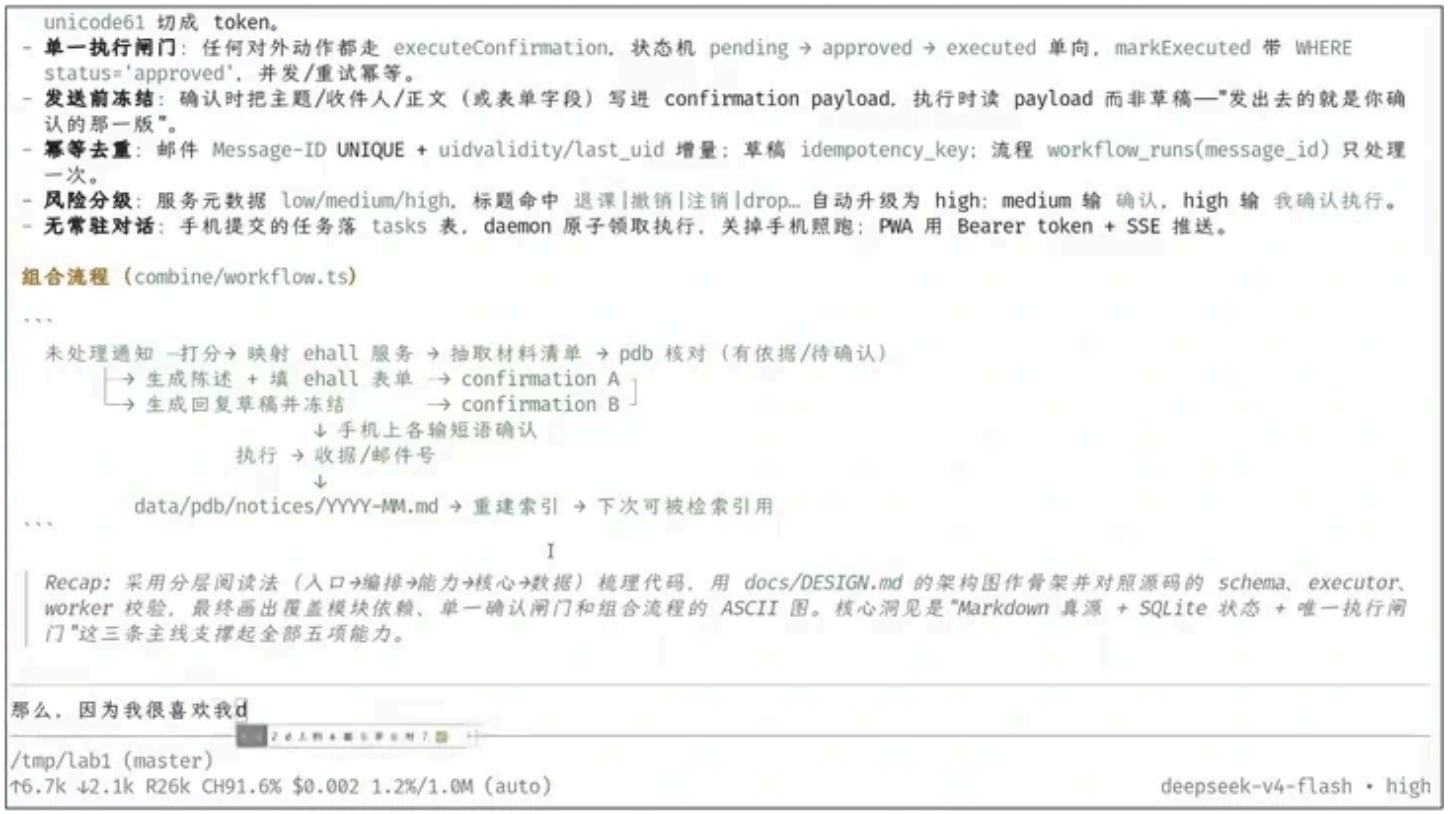
最后讲师追加需求：邮件、资料等希望与喜欢的云盘保持同步。他没有预先断言当前架构一定失败，而是让 Agent 看需要改哪些地方。观察一次变更如何穿过系统，比只看第一版结构图更能说明架构是否适合未来。

屏幕中的模型方案**区分了数据**：个人 Markdown 资料可**双向同步**；邮件按 Message-ID 命名单向归档到云；索引可从资料重建，不必同步；草稿、确认、任务与审计可提供只读镜像；IMAP 游标留在设备本地；凭据不上云。讲师当时判断此次改动看起来暂时不会引起特别大的问题。



视频原帧 97:00 原视频：云同步数据分类表，区分资料、邮件、索引、状态、游标与凭据。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:37:00 · 97:00 原视频：云同步数据分类表，区分资料、邮件、索引、状态、游标与凭据。（右侧屏幕裁切；原始全画幅留存于证据目录。）



视频原帧 96:00 原视频：模型展示确认执行、幂等与工作流设计说明，未独立验证实现。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:36:00 · 96:00 原视频：模型展示确认执行、幂等与工作流设计说明，未独立验证实现。（右侧屏幕裁切；原始全画幅留存于证据目录。）

> **整理者说明**
>
> 画面原话中的“SQLite 是各设备从记录派生的本地缓存”是这次模型提出的目标原则，但表格同时保留状态数据镜像；不能据此断言现有项目已实现完整事件重放或可靠多端同步。

画面继续提出**多设备写冲突**：两台设备同时向月度文件追加记录可能互相覆盖，可改成一事一文件，以 confirmationId 命名；也可考虑**单写者租约**或按设备分片。挂载云盘目录改动少、WebDAV 可以做更精确比对、直接网盘 API 的冲突处理取决于元数据能力。这些是模型给出的取舍，课堂没有完成实现和双设备验证。



需要改的具体位置

1. **配置**：src/core/config.ts 加 cloud: { enabled, transport, root, remoteDir, pollMs }；pdbDir 允许指向云盘挂载点。
2. **新增 src/core/db.ts 表**：cloud_state(path, hash, etag, mtime, dir, updated_at)—**存同步游标，本地库**，别放云上。
3. **src/combine/archive.ts 必须改**：现在 archiveEntry 是 **append 到 notices/YYYY-MM.md**。两台设备同时归档必然冲突。改成 **一事一文件** notices/2025-09/<confirmationId>.md，用确认 ID 命名 → 确定性 + 幂等 + 天然无冲突，还能被 pdb 当独立文档建索引。
4. **新增 src/cloud/exporter.ts**：把 smail 收到的信写成 mail/<folder>/<messageId>.eml + blobs/，命名用 Message-ID（已有 UNIQUE 约束思路），重跑幂等。
5. **新增 src/daemon tick**：cloud.sync() 后若 pdb 有新文件 → reindex；再用 SSE 通知手机端。
6. **多设备写权限**：单写者租约（云上一个 locks/daemon.lock + 心跳），或按设备分片 notices/<deviceId>/，读全部、只写自己—后者更简单更稳。

三种落地的难度

方案	改动	冲突处理
A. 挂载目录（坚果云/iCloud/OneDrive 客户端 + pdbDir 指向它）	几乎零代码，加个 fs.watch 触发 reindex	云客户端自己做冲突副本，够用
B. WebDAV 适配器（坚果云/Nextcloud）	src/cloud/ 一层，etag 尚可信	可做精确三方比对，推荐
C. 网盘 API（百度/阿里/OneDrive Graph）	OAuth + 分片上传，无稳定 etag	冲突检测难，只适合「单向备份」而非双向同步

一句话建议

分三步走：**第一步只改 pdbDir + watcher（当天可用）；第二步抽 src/cloud/ 并改 archive 为一事一文件（解决冲突这个真痛点）；第三步加 mail exporter**。顺序很重要—先立「SQLite 不出本机 + 一事一文件」两条规矩，否则云盘会让幂等设计从内部崩掉。

视频原帧 98:00 原视频：追加文件冲突、单写者/分片及三种云盘接入方案。（右侧屏幕裁切；原始全画幅留存于证据目录。）

视频原帧 01:38:00 · 98:00 原视频：追加文件冲突、单写者/分片及三种云盘接入方案。（右侧屏幕裁切；原始全画幅留存于证据目录。）

结尾回到早期决定：直接粘贴完整需求，和明确先做一个 MVP，会把项目带向不同路径。可以上来走大步、此后背着已有决定不断加功能，也可以先找到贴合核心需求的精巧表示，再逐步扩展。此前用 Git 保存学生探索历史，就是后一思路的例子；遇到真实瓶颈前不必急着换数据库。



资料图 · 第 25 页 早期 Prompt 影响方向；可用版本和可回查历史帮助重新检查设计。

官方网页讲义截图汇编（非官方原生 PDF） · PDF物理页 25 · 原始讲义 · 早期 Prompt 影响方向；可用版本和可回查历史帮助重新检查设计。

> **整理者说明**
> 本课提到自动提交与 push 是教学工作流建议，不构成本次整理对用户仓库执行提交或推送的授权。

自测：先回答，再核对

先用自己的话回答，再展开参考答案；答不清楚时，回到对应知识点。

问题 01

文档、代码、测试都在仓库中，为什么仍会失去可追溯性?

参考答案 · 核对思路

文件存在不等于它们的语义依赖被记录。链表改树后，若不知道相关文档与测试有哪些，就会形成不一致。

回查第 2 节 →

问题 02

jj 的 change ID 与 commit ID 分别解决什么?

参考答案 · 核对思路

commit ID标识一个具体提交；change ID通常在重写中延续，用于追踪同一项工作的演化。change是概念，官方模型中它不是独立对象。

回查第 6 节 →

问题 03

worktree解决了哪些并行问题，又留下哪些?

参考答案 · 核对思路

它隔离工作文件、HEAD和index，共享仓库对象与常规引用；仍需整合提交、处理语义冲突和验证，不提供容器级安全隔离。

回查第 9 节 →

问题 04

“先上锁”为什么不能自动让多Agent开发正确?

参考答案 · 核对思路

锁范围可能遗漏依赖，参与者可能不遵守协议，持锁工作也可能逻辑错误；仍需读最新状态、验证、快照和恢复策略。

回查第 11 节 →

问题 05

为什么单向汉诺塔不能直接套2^n−1?

参考答案 · 核对思路

只能沿A→B→C→A移动，最大的盘子可能需经过中间柱，递归结构改变。n=3从A到C的本例是21步。

回查第 13 节 →

问题 06

demo里出现sent或receipt，为什么不能直接宣称真实系统接通?

参考答案 · 核对思路

画面明确说明外部数据使用fixtures，真实授权和适配器最后一公里未验证。需要区分模拟流程、真实代码与真实外部执行。

回查第 17 节 →

问题 07

怎样判断一个MVP值得做?

参考答案 · 核对思路

先说清最重要的用户需要和最不确定的假设，再做能产生有效观察的最小版本；功能齐全不等于用户真正需要。

回查第 18 节 →